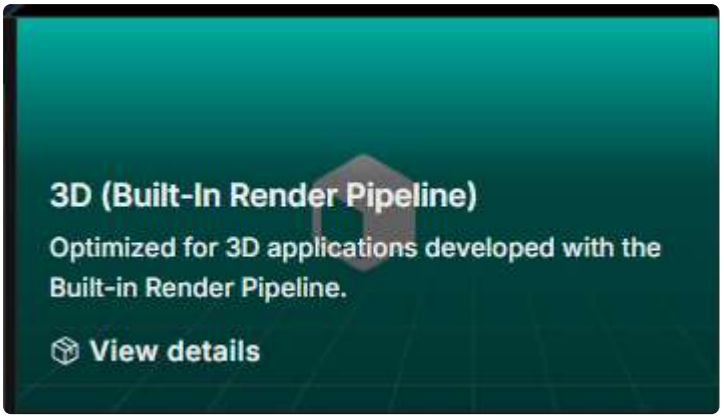
Тренировочные задания Unity

Тренировочное адание 1. 3D-игровая сцена «Сбор предметов»

Создать простую 3D-игру в Unity, где игрок управляет персонажем, собирает предметы на сцене, видит количество собранных предметов на экране, а после сбора всех предметов появляется сообщение о победе.

Создание проекта



1. Откройте **Unity Hub**.
2. Нажмите **New project**.
3. Выберите шаблон:

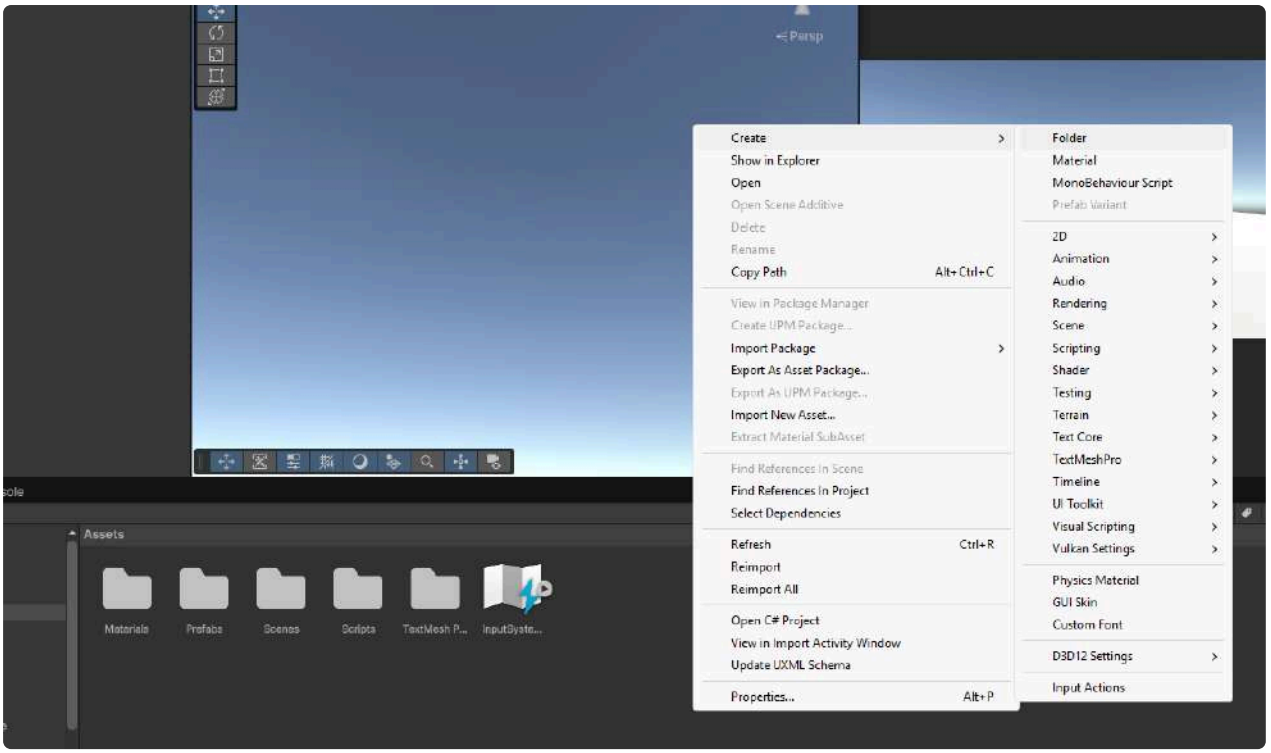
3D Built-in Render Pipeline

4. Назовите проект
5. Выберите папку для сохранения проекта.
6. Нажмите **Create project**.

В окне Project, в папке Assets. создайте папки:

- **Scenes**
- **Scripts**
- **Prefabs**

- Materials



Для создания папки:

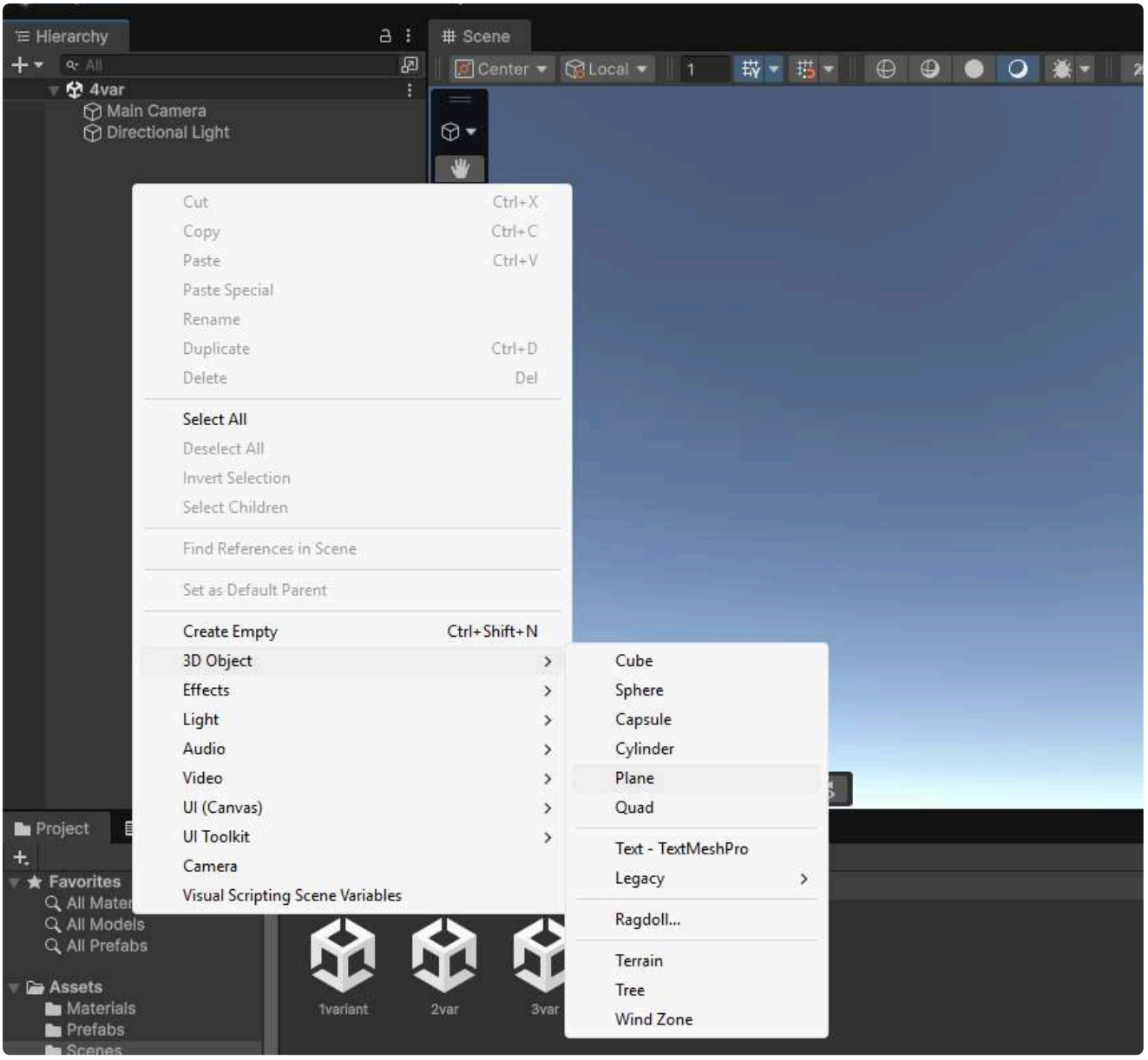
ПКМ в окне **Project** → **Create** → **Folder**

Перейдите в папку **Scenes**.
Переименуйте сцену. На экзамене можно будет назвать номером билета. Перейдите в сцену.

Создание пола

1. ПКМ по пустой области в иерархии (в области слева, где у нас все объекты):

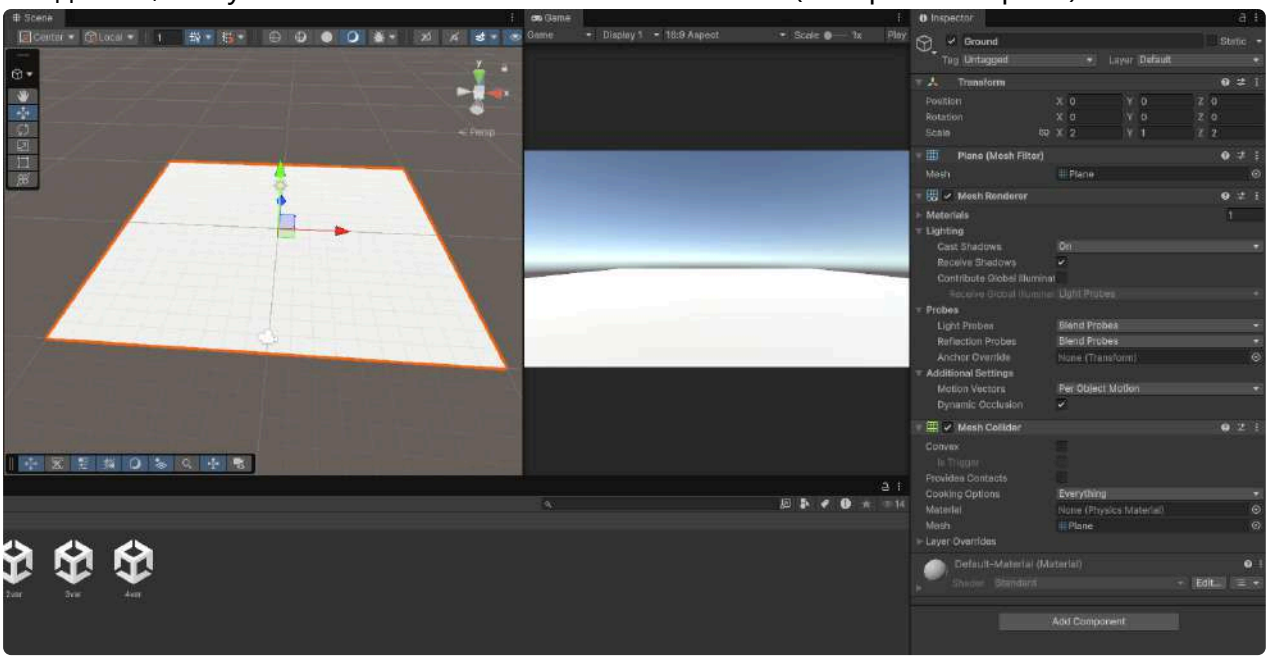
GameObject → 3D Object → Plane



2. Переименуйте объект в **Ground**.
3. В окне **Inspector** настройте Transform:

```
    Position: X = 0, Y = 0, Z = 0 (старайтесь все новые объекты ставить на нули)
Scale: X = 2, Y = 1, Z = 2 (это размеры)
```

4. Убедитесь, что у Plane есть компонент **Mesh Collider**. (в Inspector справа)



Обычно он добавляется автоматически.

Создание игрока

1. В меню выберите:

GameObject → 3D Object → Capsule

или

GameObject → 3D Object → Cube

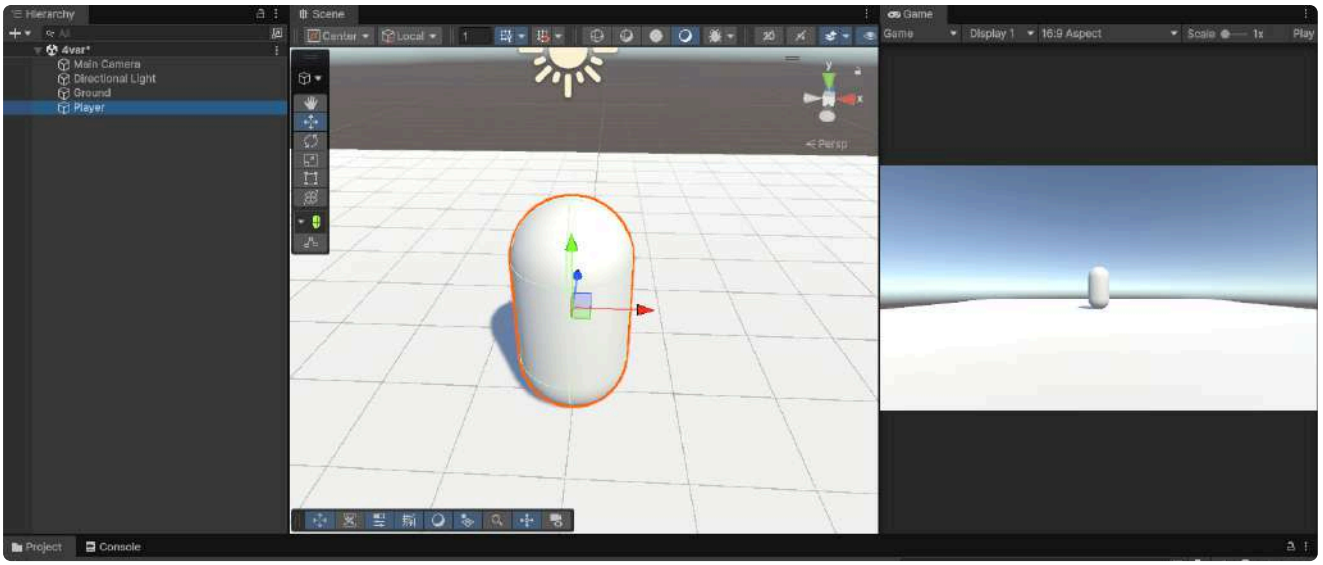
Лучше использовать **Capsule**, потому что она похожа на персонажа.

2. Переименуйте объект в: **Player**
3. Настройте Transform:

```
Position: X = 0, Y = 1, Z = 0
Scale: X = 1, Y = 1, Z = 1
```

4. У игрока должен быть компонент **Capsule Collider**.

Он добавляется автоматически.



Добавление Rigidbody игроку

1. Выделите объект **Player**.
2. В окне **Inspector** нажмите:

Add Component

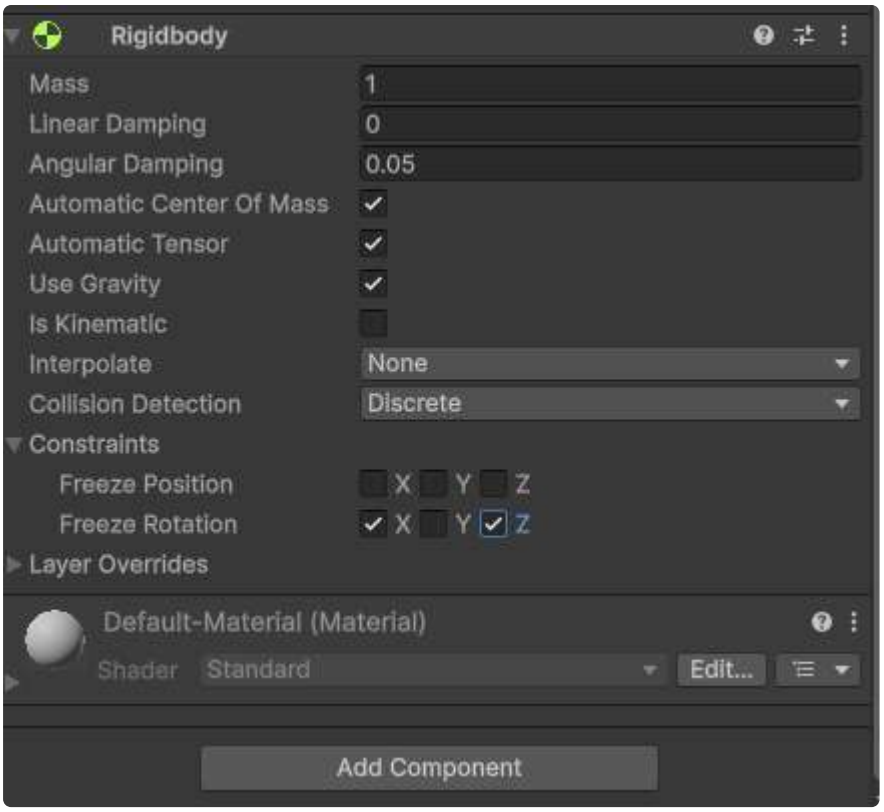
3. Найдите и добавьте:

Rigidbody

4. В компоненте Rigidbody можно включить ограничения вращения, чтобы игрок не падал на бок.

В разделе **Constraints** поставьте галочки:

```
Freeze Rotation X
Freeze Rotation Z
```



Так игрок не будет заваливаться как неваляшка при столкновениях.

Создание скрипта движения игрока

1. Нажмите в поле иерархии на Player
2. В Inspector справа, в самом низу нажмите кнопку Add Component
3. Начните вводить название скрипта:

PlayerMovement

4. Нажмите Create and Add
5. Сохраните его в вашу созданную папку Scripts
6. Откройте скрипт в Visual Studio

```
using UnityEngine;

Скрипт Unity (ссылок на ресурсы: 3) | Ссылок: 0
public class PlayerMovement : MonoBehaviour
{
    public float speed = 5f;

    private Rigidbody rb;

    Сообщение Unity | Ссылок: 0
    private void Start()
    {
        rb = GetComponent<Rigidbody>();
    }

    Сообщение Unity | Ссылок: 0
    private void Update()
    {
        Vector3 movement = Vector3.zero;

        if (Input.GetKey(KeyCode.W))
            movement += Vector3.forward;

        if (Input.GetKey(KeyCode.S))
            movement += Vector3.back;

        if (Input.GetKey(KeyCode.D))
            movement += Vector3.right;

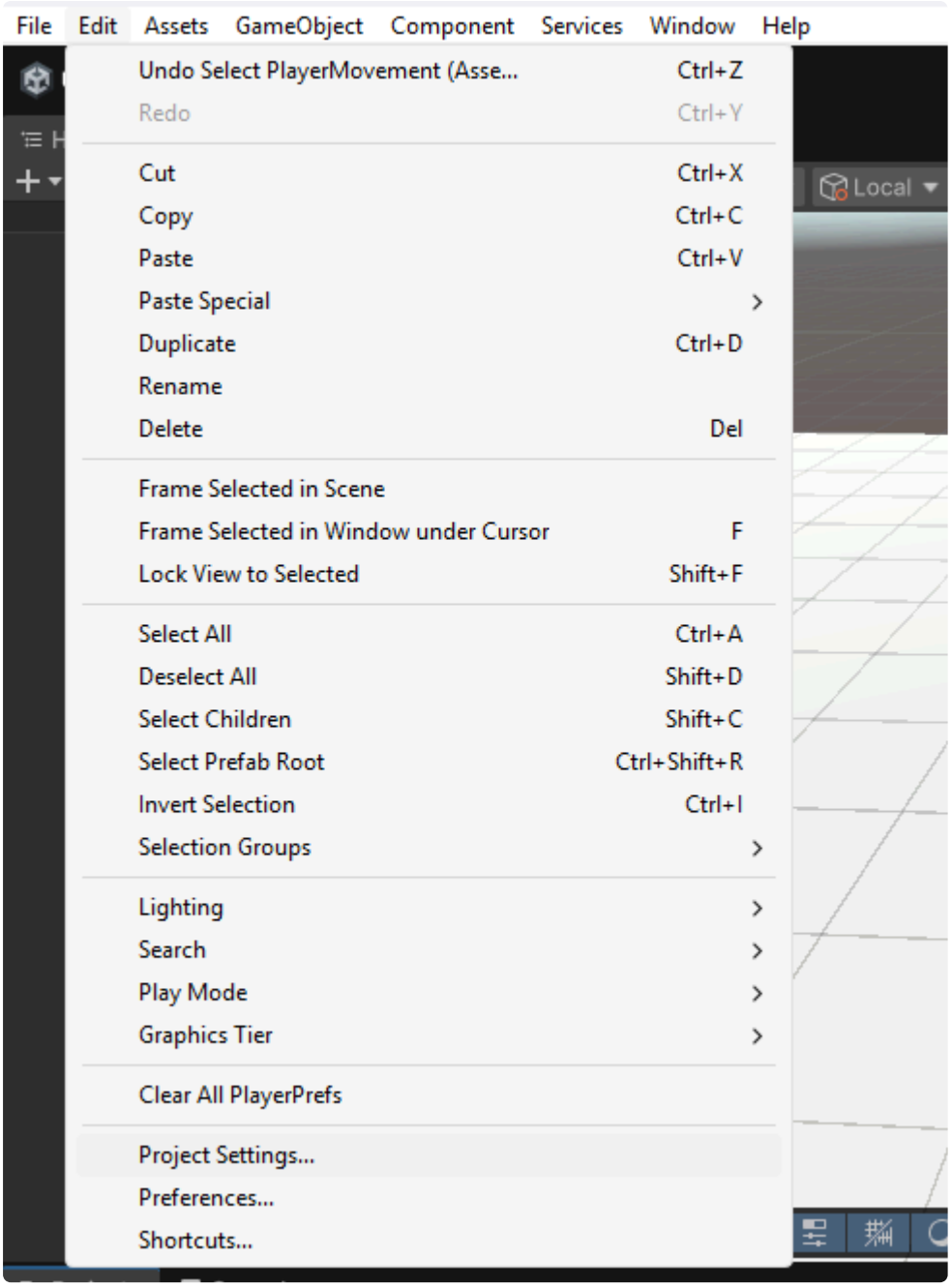
        if (Input.GetKey(KeyCode.A))
            movement += Vector3.left;

        rb.MovePosition(rb.position+movement.normalized * speed * Time.fixedDeltaTime);
    }
}
```

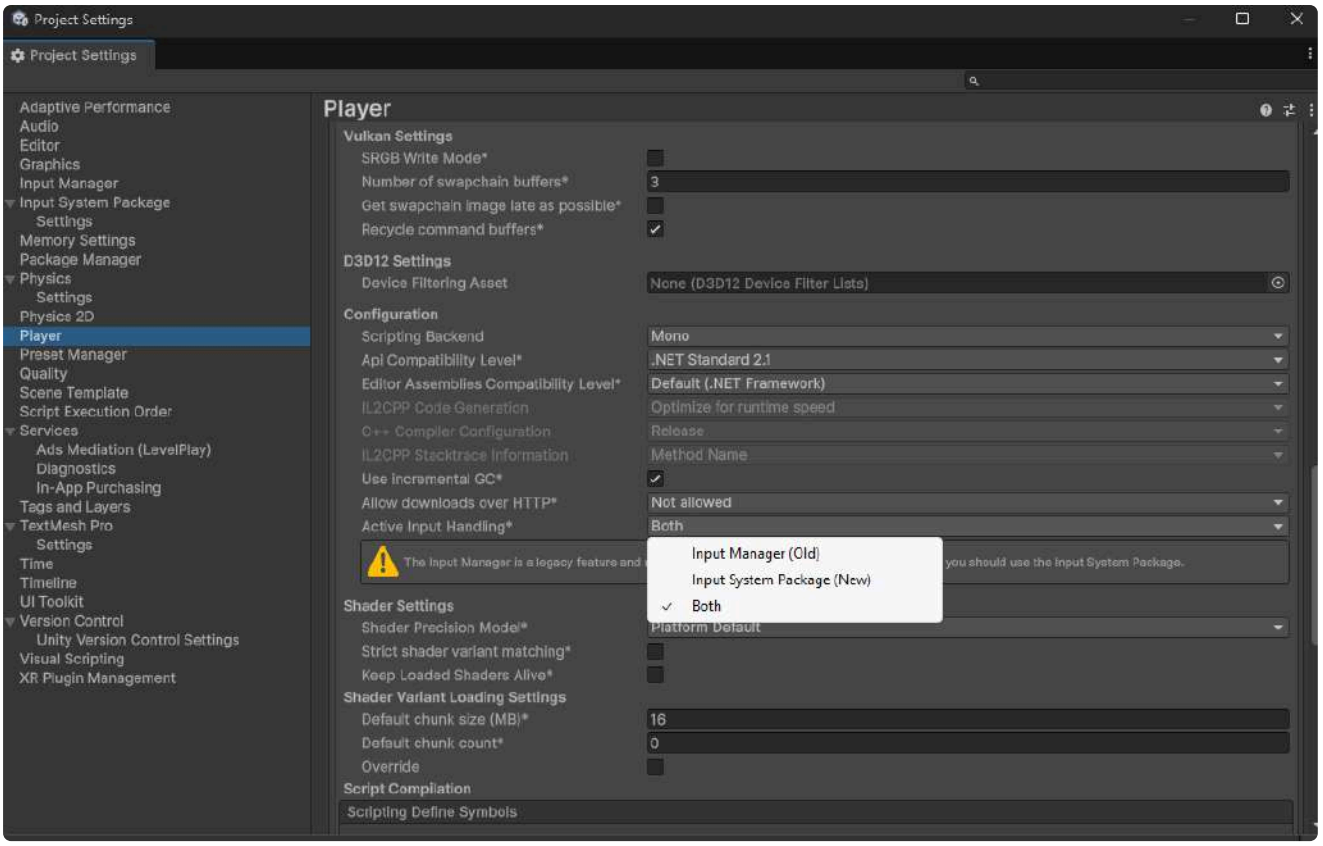
Так выглядит код на передвижение игрока.

Если возникли ошибки в Unity, необходимо в самом Unity изменить Input систему на передвижение. Для этого:

Нажимаем Edit - Project Settings



Ищем слева вкладку Player, внутри Other Setting, листаем до Configuration - в Active Input Handling ставим BOTH



После этого проект перезагружается.

Теперь наш персонаж может двигаться.

Настройка камеры

1. Выделите объект **Main Camera**.
2. Настройте Transform примерно так:

```
Position: X = 0, Y = 8, Z = -8
Rotation: X = 45, Y = 0, Z = 0
```

Так камера будет смотреть на игровое поле сверху под углом. Либо можете выставить камеру самостоятельно.

Создание предмета для сбора

1. Выберите:

GameObject → 3D Object → Sphere

2. Переименуйте объект в:

Collectible или же **Ball**

3. Настройте Transform:

```
Position: X = 2, Y = 0.5, Z = 2
Scale: X = 0.5, Y = 0.5, Z = 0.5
```

4. У Sphere автоматически будет компонент **Sphere Collider**.
5. В компоненте **Sphere Collider** поставьте галочку:

```
Is Trigger
```

Это нужно, чтобы предмет не толкал игрока, а просто засчитывался при касании.

Создание материала для предмета

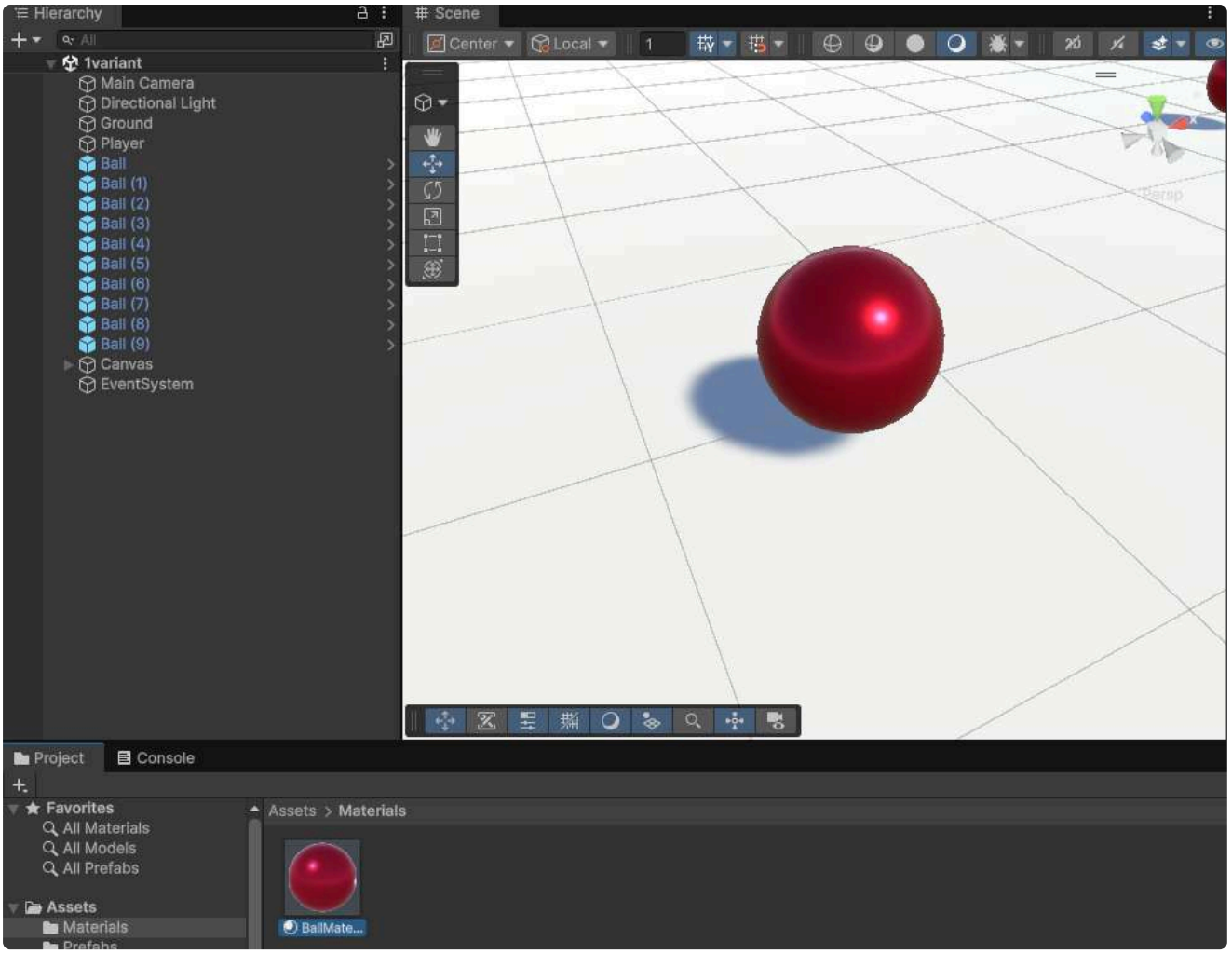
1. Откройте папку **Materials**.
2. Нажмите ПКМ:

Create → Material

3. Назовите материал:

BallMaterial

4. В Inspector выберите цвет, например красный.
5. Перетащите материал на объект **Collectible (Ball)**.



Создание префаба предмета

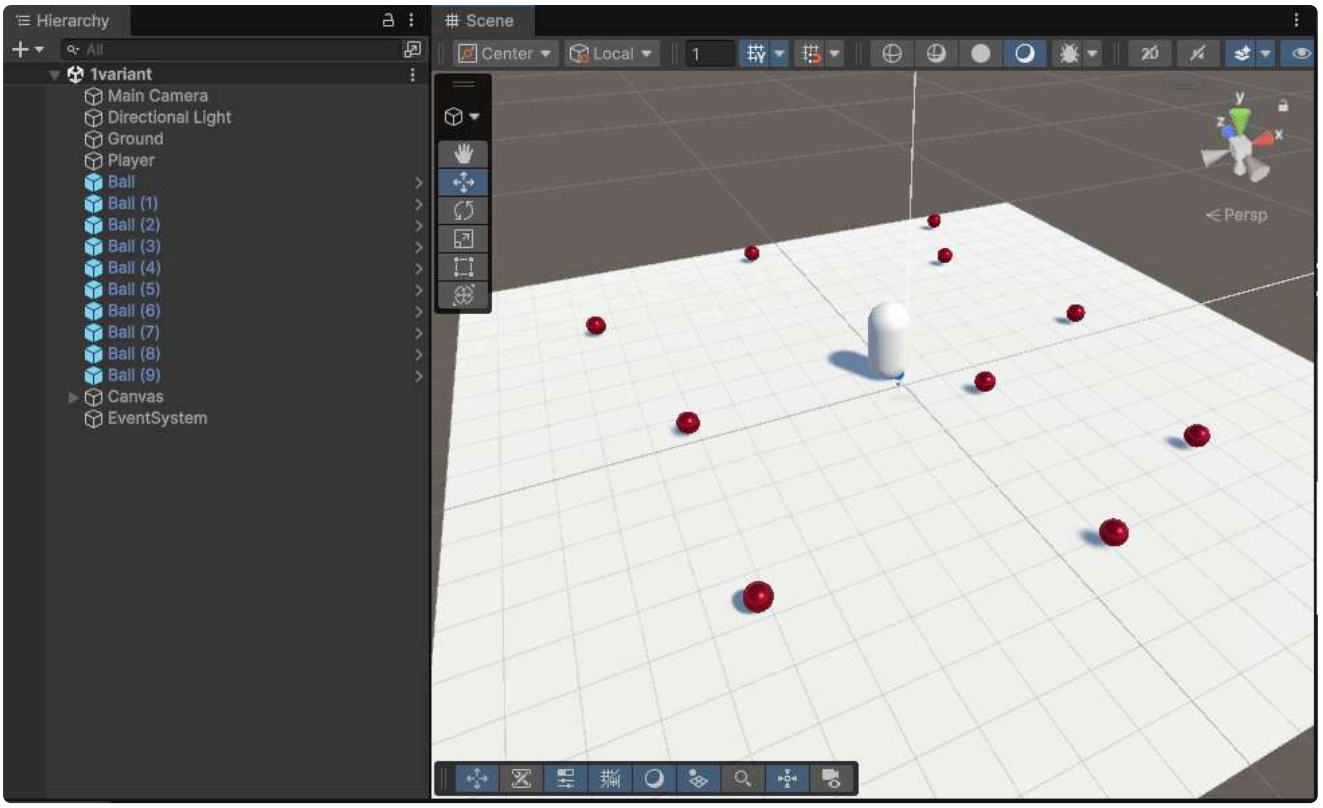
1. Откройте папку **Prefabs**.
2. Перетащите объект **Collectible(Ball)** из окна **Hierarchy** в папку **Prefabs**.

Теперь это префаб.

3. Объект на сцене можно оставить.
4. Создайте копии предмета:

Выделите **Collectible(Ball)** → нажмите **Ctrl + D**

5. Разместите минимум 10 предметов по сцене.



Создание UI для отображения очков

1. Выберите:

GameObject → UI(Canvas) → Text - TextMeshPro

Если появится окно импорта TMP Essentials, нажмите:

Import TMP Essentials

2. Переименуйте объект в:

ScoreText

3. В Inspector настройте текст:

```
Собрано: 0
```

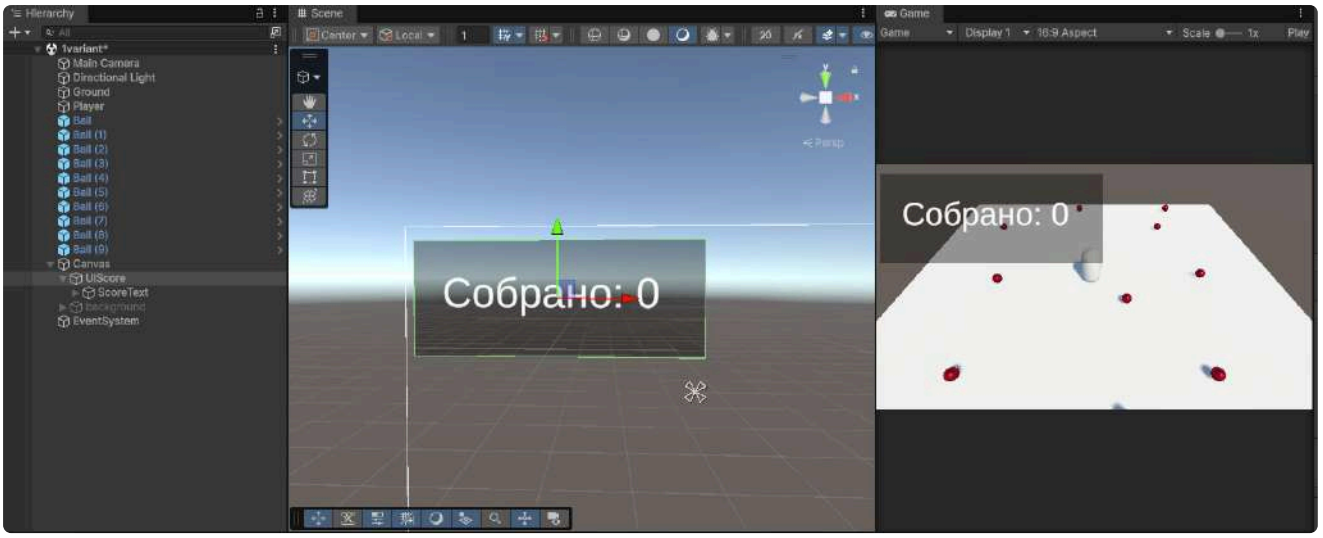
Задайте цвет тексту, размер шрифта, расположение по середине.

Также создайте фон тексту.
ПКМ по Canvas - UI(Canvas) - Image
Переименуйте Image в UIScore

Переместите ScoreText вовнутрь UIScore

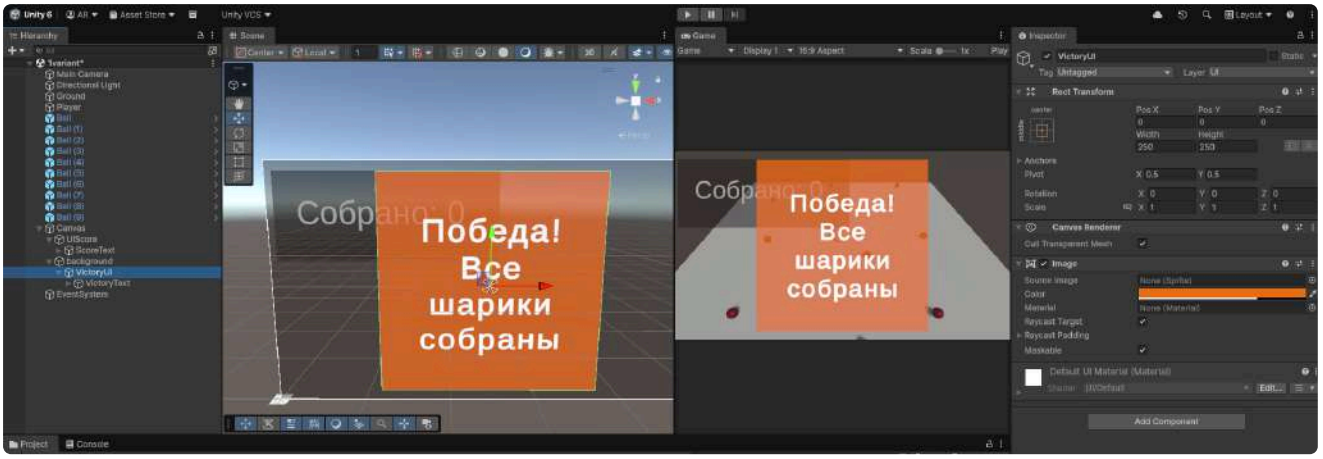


Тексту и Image поставьте позиции на нули в инспекторе справа. Чтобы они были друг на друге. Настройте ширину и высоту для Image, покрасьте и повысьте прозрачность.



Теперь нажав на UIScore вы перетаскиваете два взаимосвязанных объекта. Перетащите например в левый верхний угол по канвасу.

Подобным образом создайте UI для отображения сообщения о победе.



По объектам: VictoryText - это TextMeshPro
VictoryUI - это оранжевый Image
Background - затемненный фон.

Как сделать Background:
ПКМ по Canvas - UI(Canvas) - Image

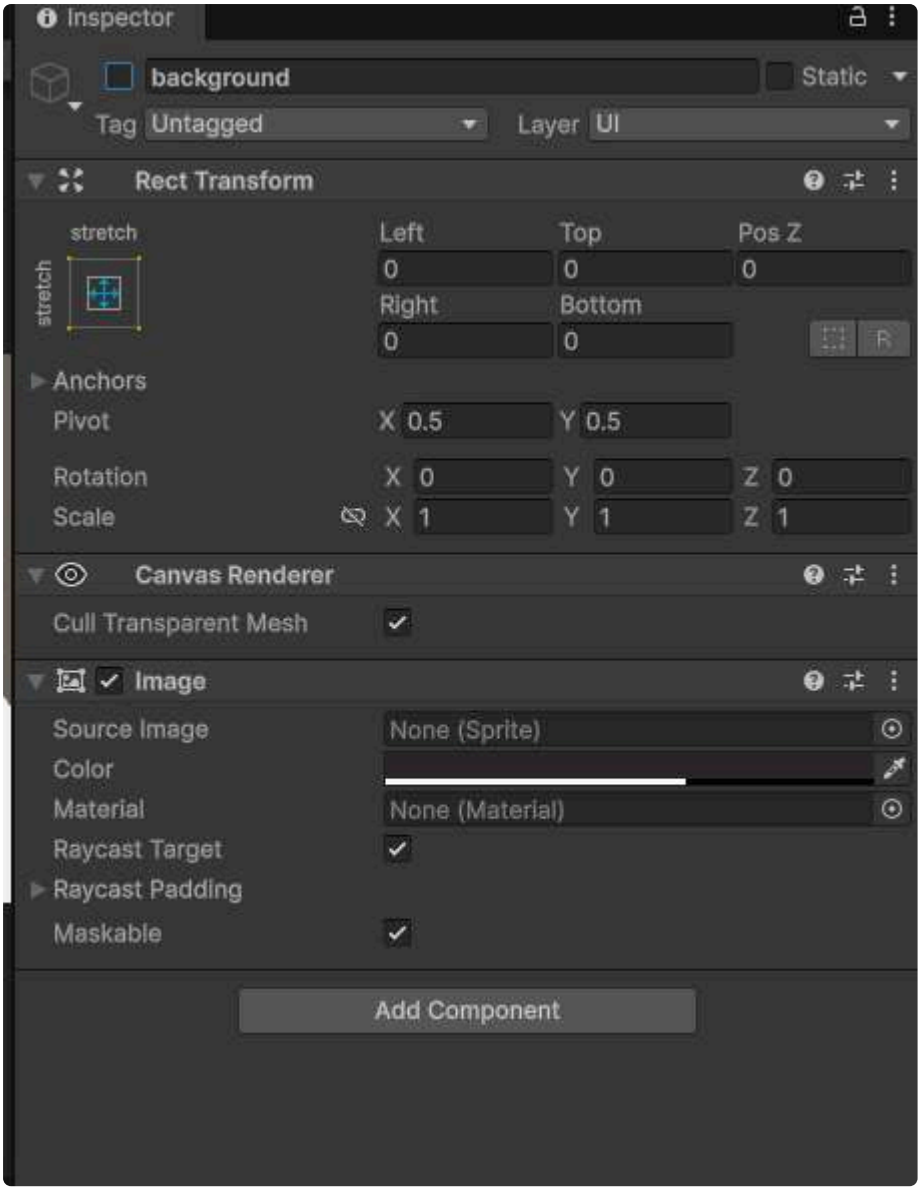
Нажмите в Inspector на данный якорь:



Зажмите комбинацию клавиш Alt+Shift и выберите нижний правый квадратик. Изображение растянется на весь экран. Покрасьте в темный и повысьте прозрачность. Background готов.

Внутри Background должен быть Victory UI, а внутри VictoryUI - VictoryText

Чтобы сообщение о победе нам пока не мешало, и в принципе оно должно выводиться только при победе - снимите галочку (сделайте объект неактивным) около названия объекта:



Создание скрипта сбора предметов

1. Нажмите на Player
2. Нажмите Add Component и начните вводить название скрипта:

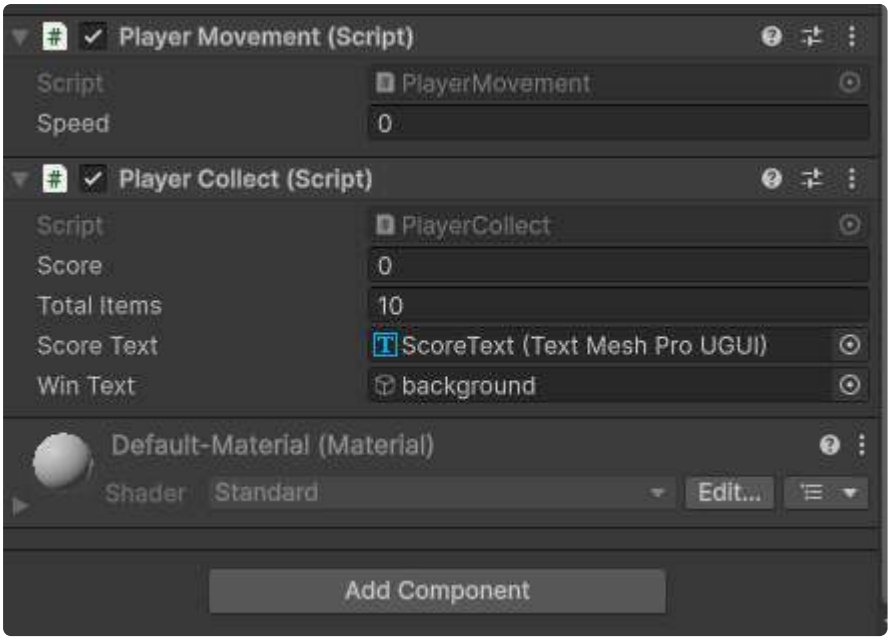
PlayerCollector

3. Откройте скрипт в Visual Studio.
4. Напишите скрипт для сбора:



Нажмите подключить к Unity или обновите код, если уже было подключено.

В Inspector у Player появится данный скрипт для сбора



Перетащите объект **ScoreText** из Hierarchy в поле **Score Text**.
Перетащите объект **Background** из Hierarchy в поле **Win Text**.
В поле **Total Items** укажите (это если у вас 10 шариков по полу):

```
10
```

Настройка тега Collectible(Ball)

1. Выделите объект **Collectible**.
2. В верхней части Inspector найдите поле **Tag**.
3. Нажмите:

Add Tag

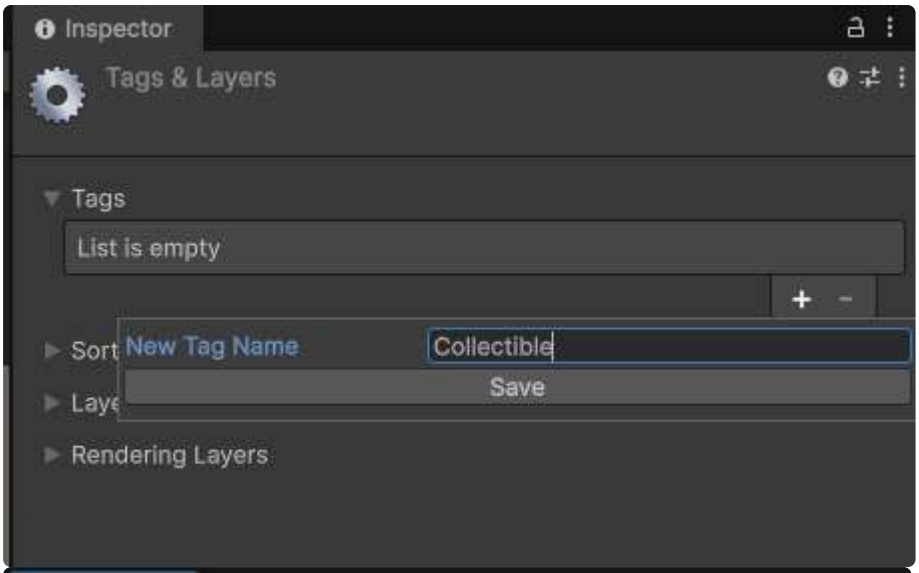
4. Нажмите плюс.
5. Создайте тег:

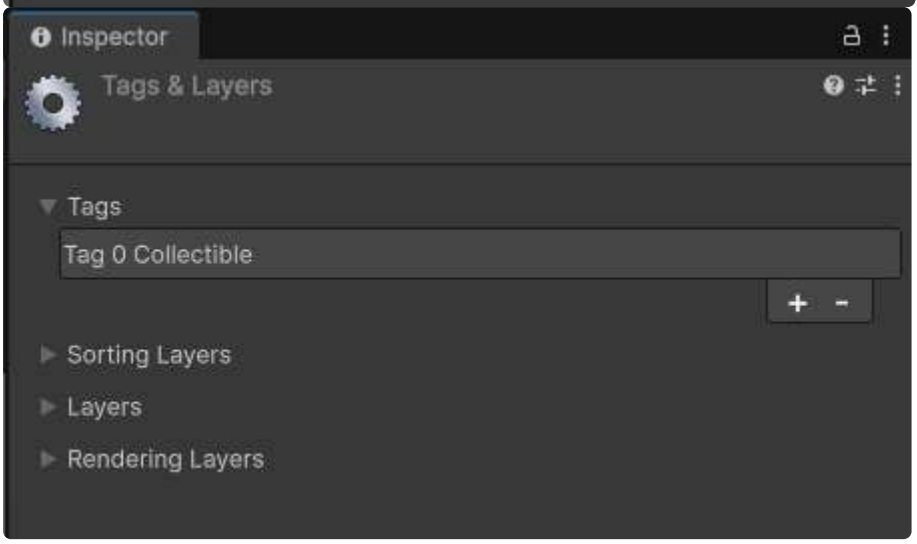
```
Collectible
```

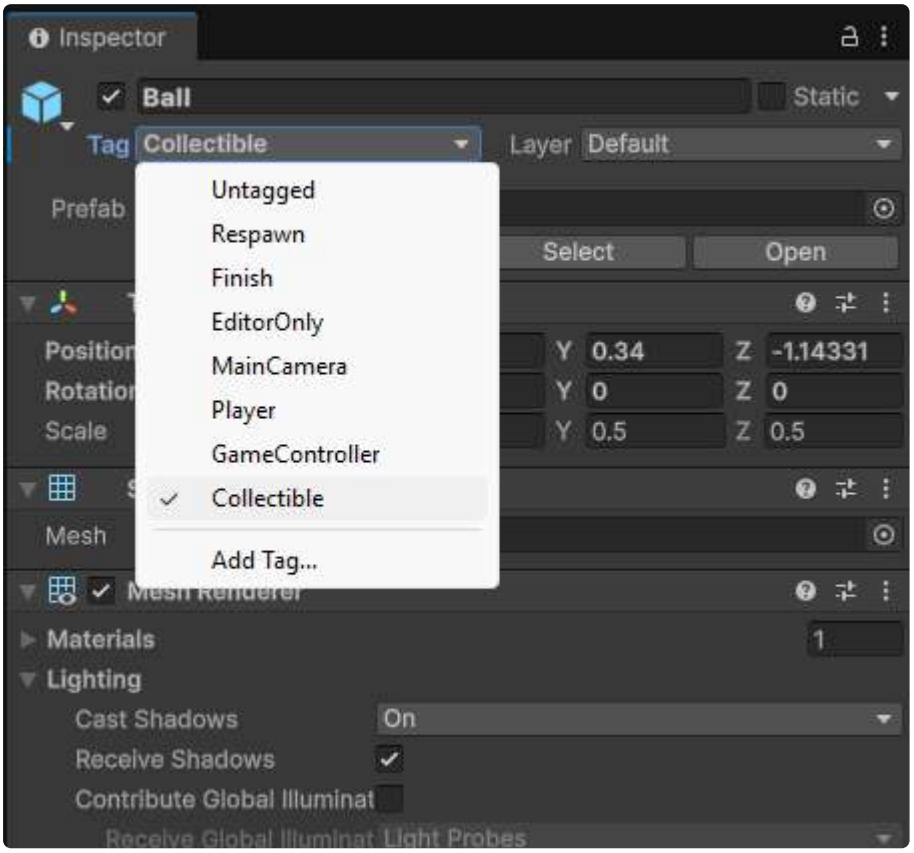
6. Вернитесь к каждому объекту для сбора.
7. Назначьте им тег:

```
Collectible
```

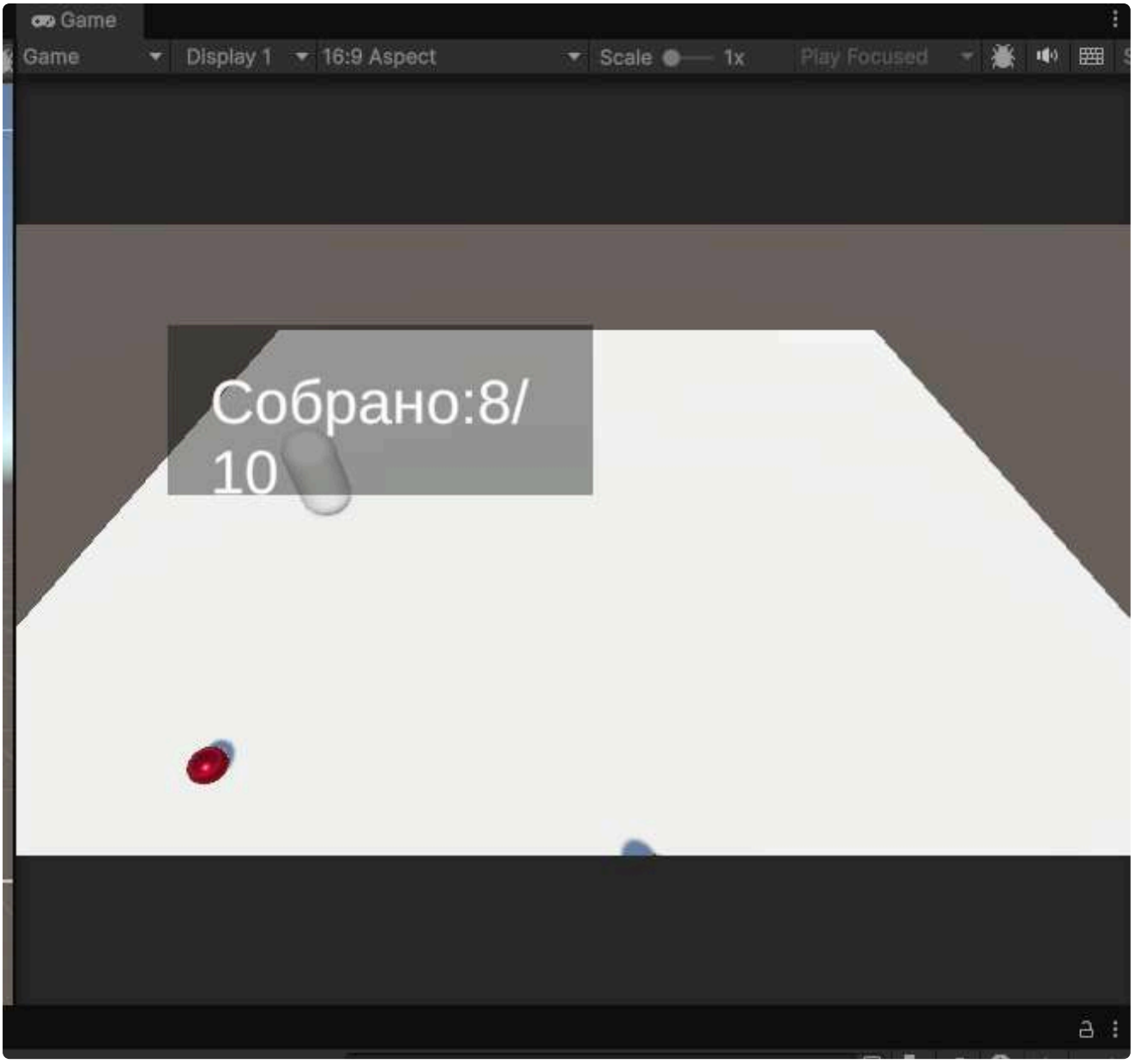
Важно: тег нужно назначить всем 10 предметам.

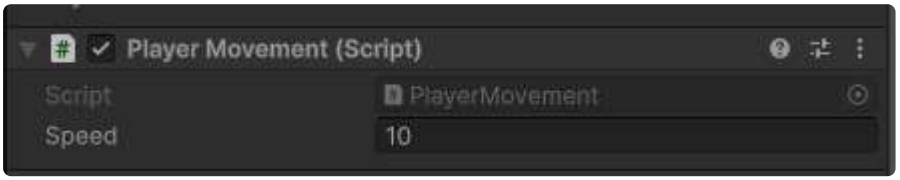

Inspector
Tags & Layers
Tags
List is empty
Sort
New Tag Name
Collectible
Save
Rendering Layers


Inspector
Tags & Layers
Tags
Tag 0 Collectible
Sorting Layers
Layers
Rendering Layers

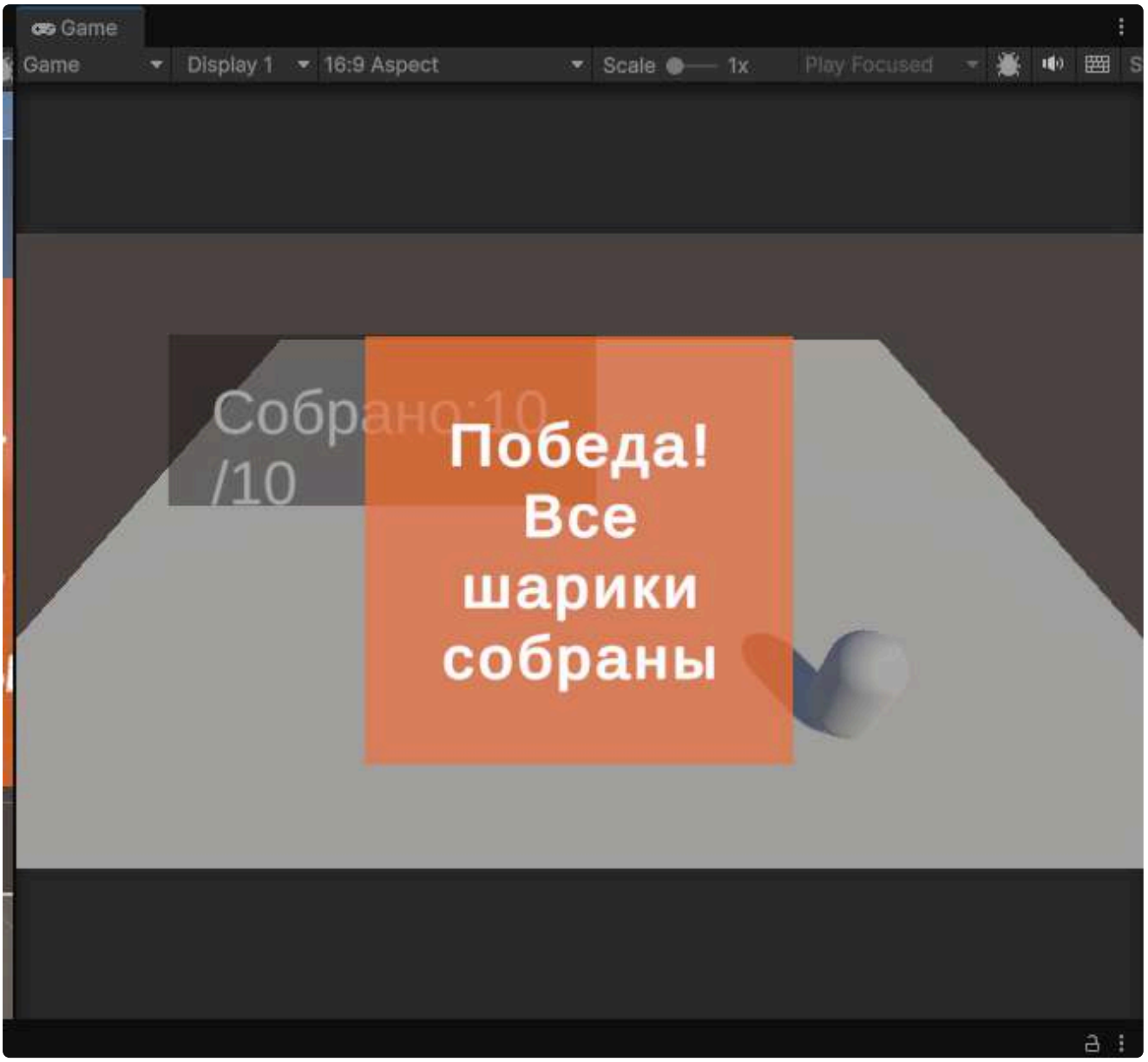

Inspector
Ball
Static
Tag
Collectible
Layer
Default
Prefab
Select
Open
Position
Rotation
Scale
Y 0.34
Z -1.14331
Y 0
Z 0
Y 0.5
Z 0.5
Untagged
Respawn
Finish
EditorOnly
MainCamera
Player
GameController
Collectible
Add Tag...
Mesh
Materials
1
Lighting
Cast Shadows
On
Receive Shadows
Contribute Global Illuminat

После этого все должно работать.





Если игрок вдруг не двигается, проверьте, чтобы здесь в Inspector в скрипте Player Movement в параметре Speed не стояло значение "0".



Тренировочное задание 2. 3D-сцена «Полоса препятствий»

Создать простую 3D-сцену, где игрок управляет персонажем, проходит препятствия, может упасть в зону проигрыша, а при достижении финиша получает сообщение о завершении уровня.

Для предыдущего тренировочного задания рассказано, как правильно создать проект и папки для работы. Лучше в папке Scenes создайте новую сцену. ПКМ - Create - Scene - Scene.

Создание уровня

Можно сделать уровень из одного **Plane** или из нескольких **Cube**. Для полосы препятствий удобнее использовать несколько кубов-платформ.

Вариант с Plane

1. Выберите:

```
GameObject → 3D Object → Plane
```

2. Переименуйте объект в:

```
Ground
```

3. Настройте Transform:

```
Position: X = 0, Y = 0, Z = 0
Scale: X = 2, Y = 1, Z = 5
```

Вариант с Cube-платформой

1. Выберите:

```
GameObject → 3D Object → Cube
```

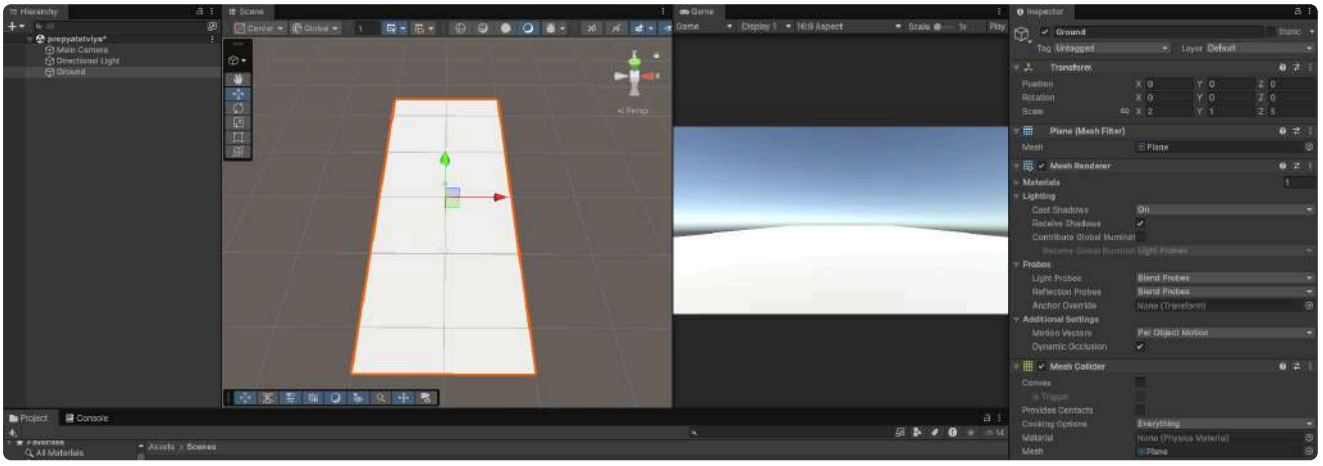
2. Переименуйте объект в:

```
Platform
```

3. Настройте Transform:

```
Position: X = 0, Y = 0, Z = 0
Scale: X = 6, Y = 0.3, Z = 20
```

4. У объекта Cube автоматически есть **Box Collider**.



Создание игрока

1. Выберите:

```
GameObject → 3D Object → Capsule
```

2. Переименуйте объект в:

```
Player
```

3. Настройте Transform:

```
Position: X = 0, Y = 1.3, Z = -8
Scale: X = 1, Y = 1, Z = 1
```

4. У игрока должен быть компонент:

```
Capsule Collider
```

Он добавляется автоматически.

Добавление Rigidbody игроку

1. Выделите объект **Player**.
2. В Inspector нажмите:

```
Add Component
```

3. Добавьте:

```
Rigidbody
```

4. Настройте Rigidbody:

```
Mass: 1Use Gravity: включеноIs Kinematic: выключено
```

5. В разделе **Constraints** включите:

```
Freeze Rotation XFreeze Rotation Z
```

Это нужно, чтобы игрок не заваливался на бок.

Создание скрипта движения игрока

Если вы уже делали первое тренировочное задание, можете взять уже готовый скрипт! Он у вас есть! Нажимая на Player - Add Component - и вводите PlayerMovement. Этот скрипт уже имеется у вас в папке Scripts

1. Нажмите в поле иерархии на Player
2. В Inspector справа, в самом низу нажмите кнопку Add Component
3. Начните вводить название скрипта:

PlayerMovement

4. Нажмите Create and Add
5. Сохраните его в вашу созданную папку Scripts
6. Откройте скрипт в Visual Studio

```
using UnityEngine;

Скрипт Unity (ссылок на ресурсы: 3) | Ссылок: 0
public class PlayerMovement : MonoBehaviour
{
    public float speed = 5f;

    private Rigidbody rb;

    Сообщение Unity | Ссылок: 0
    private void Start()
    {
        rb = GetComponent<Rigidbody>();
    }

    Сообщение Unity | Ссылок: 0
    private void Update()
    {
        Vector3 movement = Vector3.zero;

        if (Input.GetKey(KeyCode.W))
            movement += Vector3.forward;

        if (Input.GetKey(KeyCode.S))
            movement += Vector3.back;

        if (Input.GetKey(KeyCode.D))
            movement += Vector3.right;

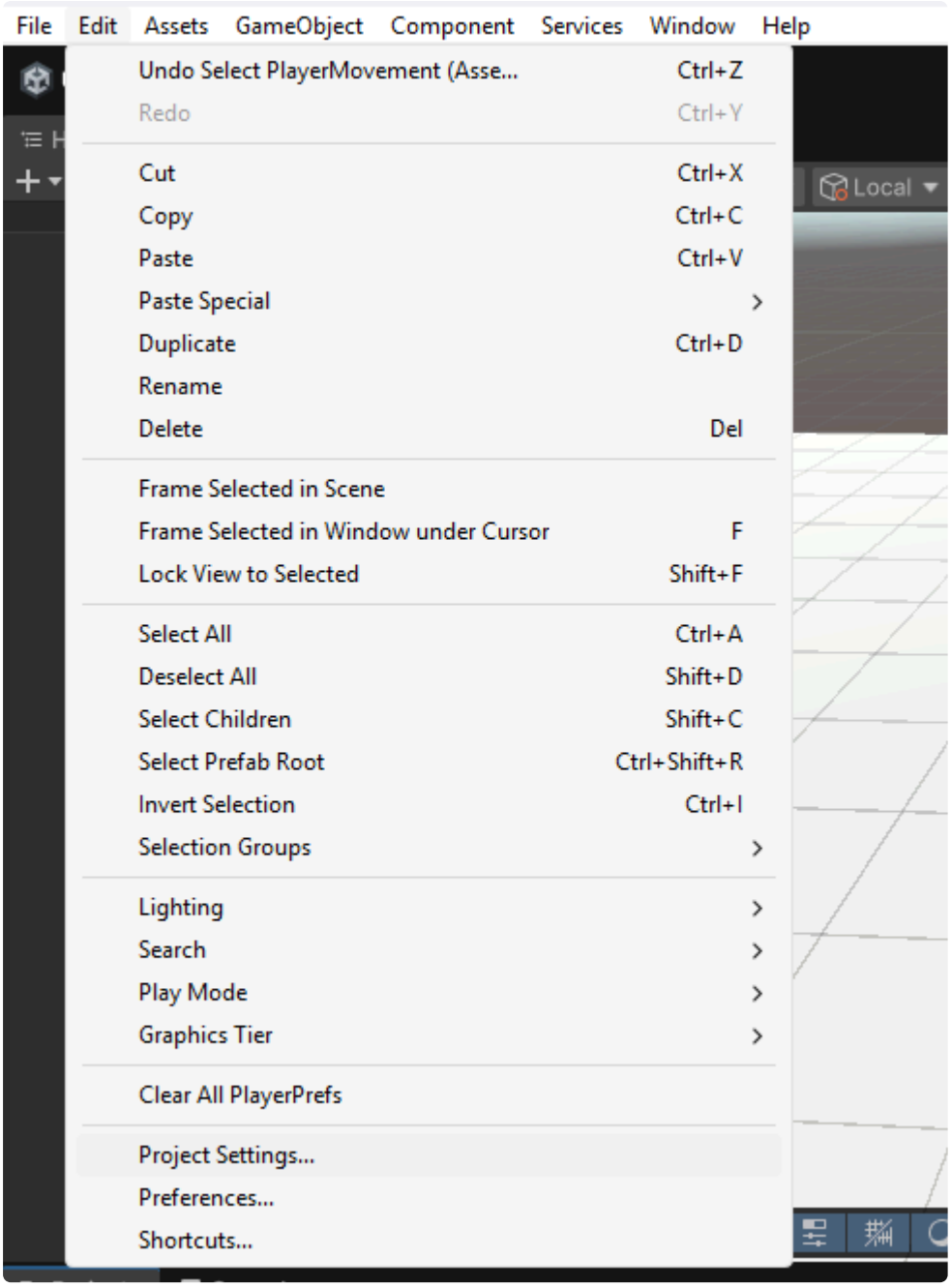
        if (Input.GetKey(KeyCode.A))
            movement += Vector3.left;

        rb.MovePosition(rb.position+movement.normalized * speed * Time.fixedDeltaTime);
    }
}
```

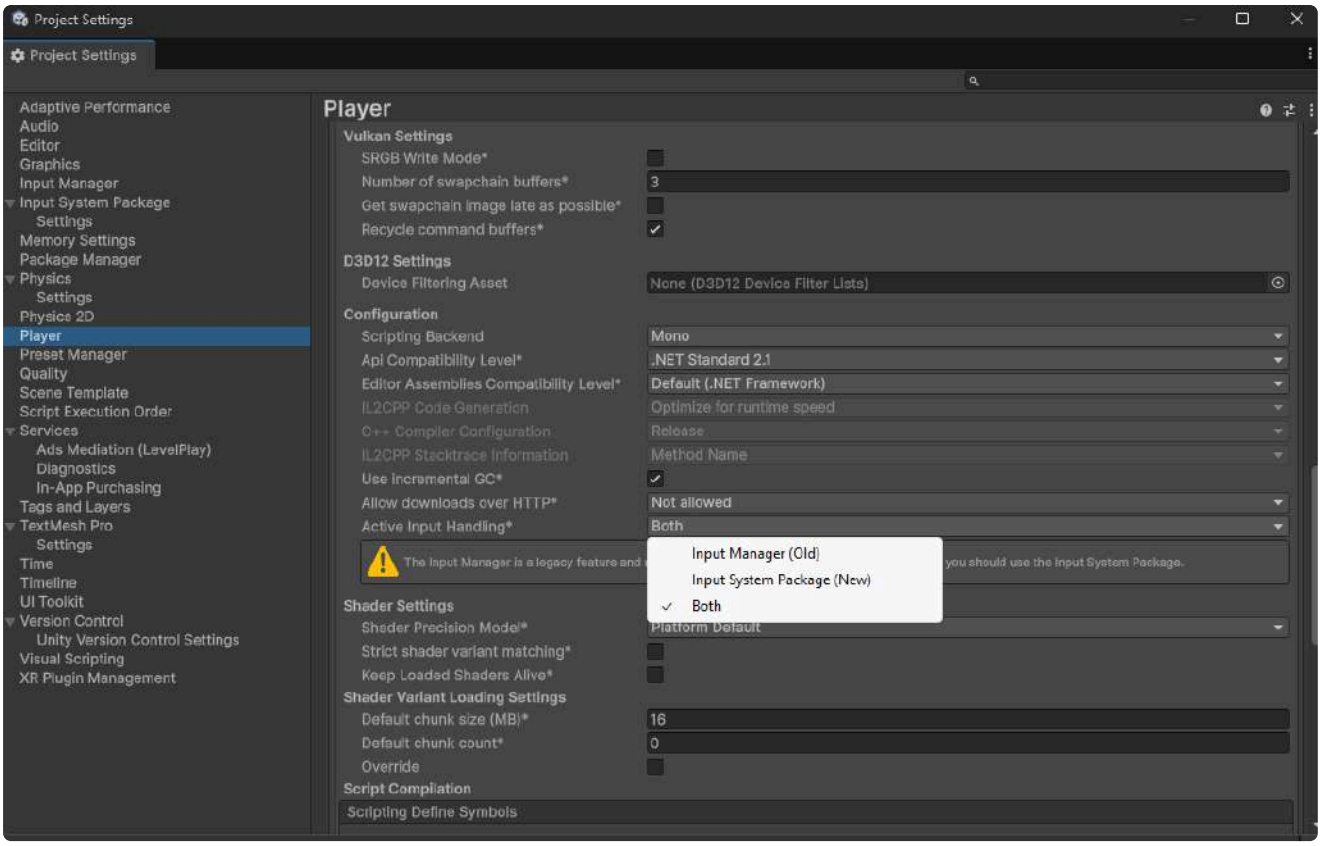
Так выглядит код на передвижение игрока.

Если возникли ошибки в Unity, необходимо в самом Unity изменить Input систему на передвижение. Для этого:

Нажимаем Edit - Project Settings



Ищем слева вкладку Player, внутри Other Setting, листаем до Configuration - в Active Input Handling ставим BOTH



После этого проект перезагружается.

Теперь наш персонаж может двигаться.

Настройка камеры

1. Выделите **Main Camera**.
2. Настройте Transform:

```
Position: X = 0, Y = 8, Z = -13
Rotation: X = 35, Y = 0, Z = 0
```

Камера будет смотреть на уровень сверху под углом.

Создание препятствий

Нужно создать минимум 5 препятствий из 3D-объектов.

Препятствие 1

1. Выберите:

```
GameObject → 3D Object → Cube
```

2. Переименуйте:

```
Obstacle_1
```

3. Настройте Transform:

Либо расставьте кубы самостоятельно как хотите.

```
Position: X = -2, Y = 0.6, Z = -4
Scale: X = 1, Y = 1, Z = 1
```

Препятствие 2

Создайте Cube и настройте:

```
Name: Obstacle_2
Position: X = 2, Y = 0.6, Z = -1
Scale: X = 1, Y = 1, Z = 1
```

Препятствие 3

```
Name: Obstacle_3
Position: X = 0, Y = 0.6, Z = 2
Scale: X = 4, Y = 1, Z = 0.5
```

Препятствие 4

```
Name: Obstacle_4
Position: X = -2, Y = 0.6, Z = 5
Scale: X = 1, Y = 1, Z = 1
```

Препятствие 5

```
Name: Obstacle_5
Position: X = 2, Y = 0.6, Z = 8
Scale: X = 1, Y = 1, Z = 1
```

У каждого Cube автоматически есть **Box Collider**, поэтому игрок не сможет проходить сквозь препятствия.

Создание зоны проигрыша

Зона проигрыша нужна, чтобы определить, что игрок упал с уровня.

1. Выберите:

```
GameObject → 3D Object → Cube
```

2. Переименуйте объект:

```
LoseZone
```

3. Настройте Transform:

```
Position: X = 0, Y = -2, Z = 0
Scale: X = 20, Y = 0.2, Z = 30
```

4. У объекта **LoseZone** в компоненте **Box Collider** поставьте галочку:

```
Is Trigger
```

5. Чтобы зона не мешала визуально, можно отключить **Mesh Renderer**:

Выделите **LoseZone** → в Inspector найдите **Mesh Renderer** → снимите галочку.

Теперь объект будет невидимым, но триггер останется работать.

Создание финиша

1. Выберите:

```
GameObject → 3D Object → Cube
```

2. Переименуйте:

```
FinishZone
```

3. Настройте Transform:

```
Position: X = 0, Y = 0.5, Z = 11
Scale: X = 5, Y = 1, Z = 0.3
```

4. В компоненте **Box Collider** включите:

```
Is Trigger
```

5. Можно изменить цвет финиша, чтобы он выделялся.

Создание материалов

Материал для пола

1. Откройте папку **Materials**.
2. Нажмите:

```
ПКМ → Create → Material
```

3. Назовите:

```
GroundMaterial
```

4. Выберите цвет.
5. Перетащите материал на объект **Ground** или **Platform**.

Материал для препятствий

1. Создайте материал:

```
ObstacleMaterial
```

2. Выберите другой цвет.
3. Перетащите его на все препятствия.

Материал для финиша

1. Создайте материал:

```
FinishMaterial
```

2. Выберите яркий цвет.
3. Перетащите его на **FinishZone**.

Создание UI для сообщений

1. Выберите:

```
GameObject → UI → Text – TextMeshPro
```

2. Если Unity предложит импортировать TMP Essentials, нажмите:

```
Import TMP Essentials
```

3. Переименуйте текстовый объект:

```
MessageText
```

4. В Inspector в поле Text напишите:

```
Доберитесь до финиша!
```

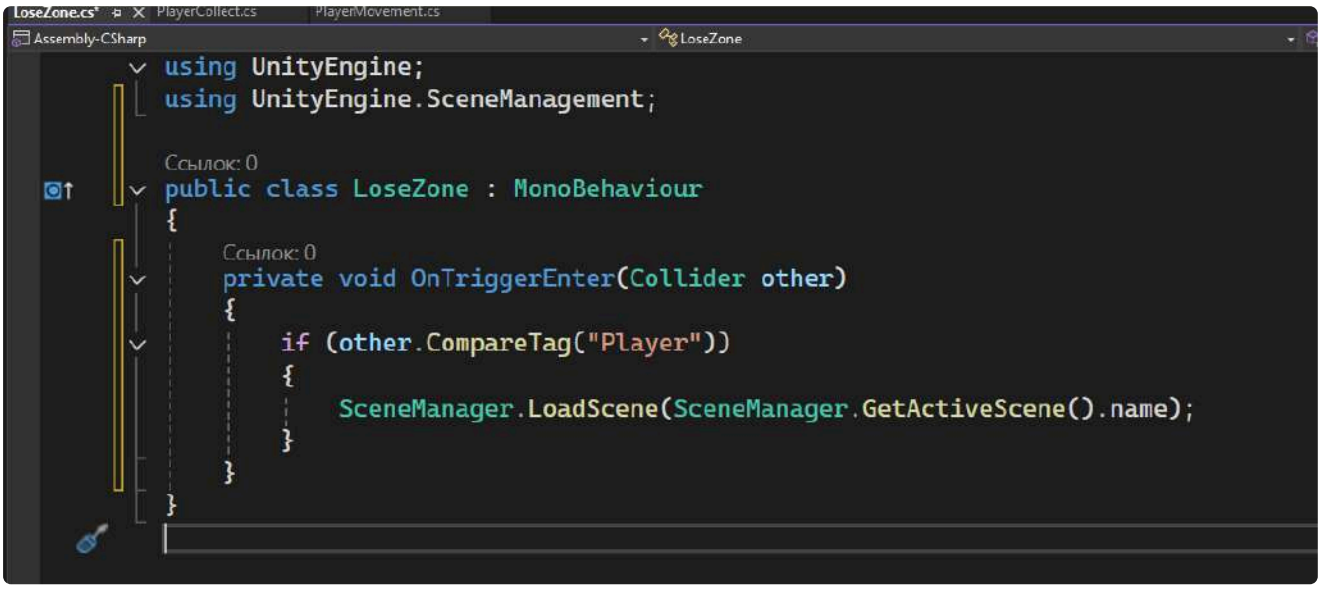
5. Настройте расположение сверху экрана. Также можете добавить фон тексту.

Создание скрипта для зоны проигрыша

1. Нажмите на объект LoseZone
2. Add Component, начните вводить название скрипта:

```
LoseZone
```

3. Откройте скрипт и напишите код:



3. Сохраните скрипт.
4. Вернитесь в Unity.

Назначение тега Player

Чтобы зона проигрыша понимала, что в неё попал именно игрок:

1. Выделите объект **Player**.
2. В Inspector сверху найдите поле **Tag**.
3. Выберите:

```
Player
```

Если тега Player нет:

```
Tag → Add Tag → + → Player
```

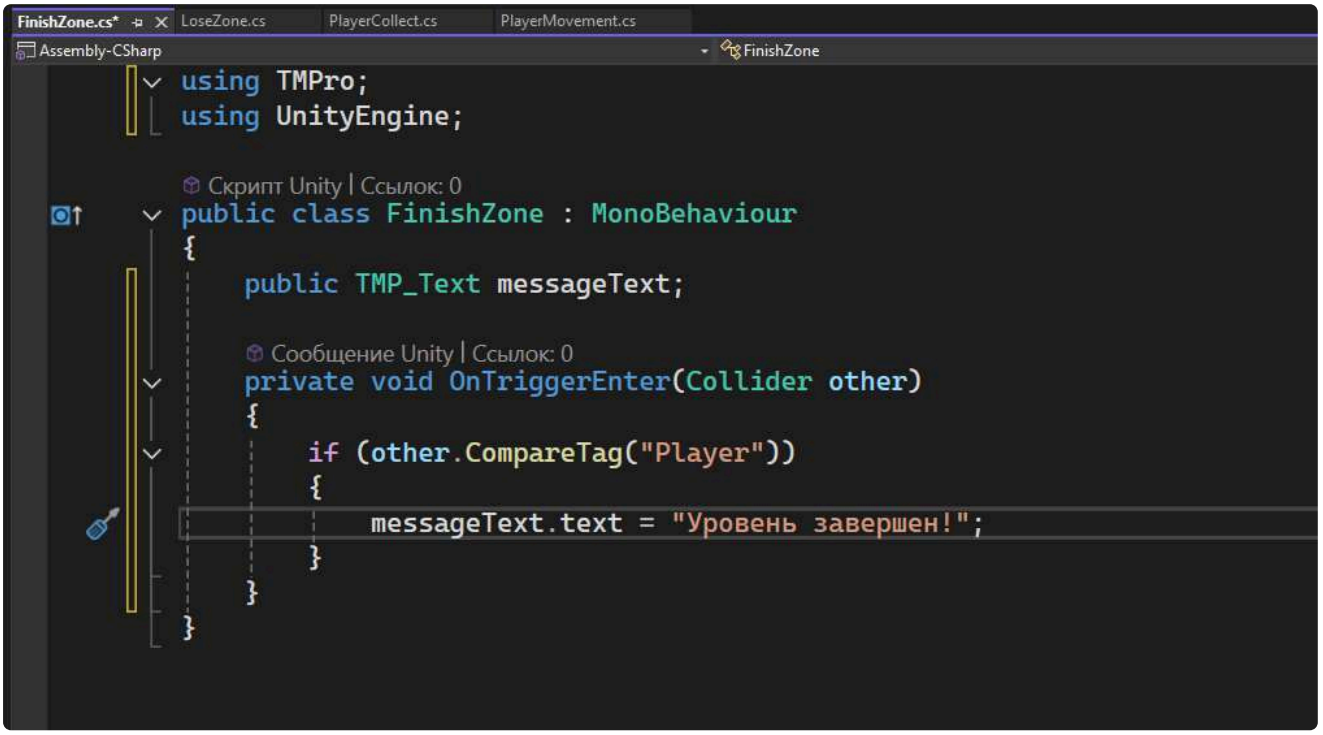
Потом снова выделите Player и назначьте ему тег **Player**.

Создание скрипта финиша

1. Нажмите на компонент FinishZone
2. Add Component - начните вводить название скрипта:

```
FinishZone
```

3. Откройте скрипт и напишите код:

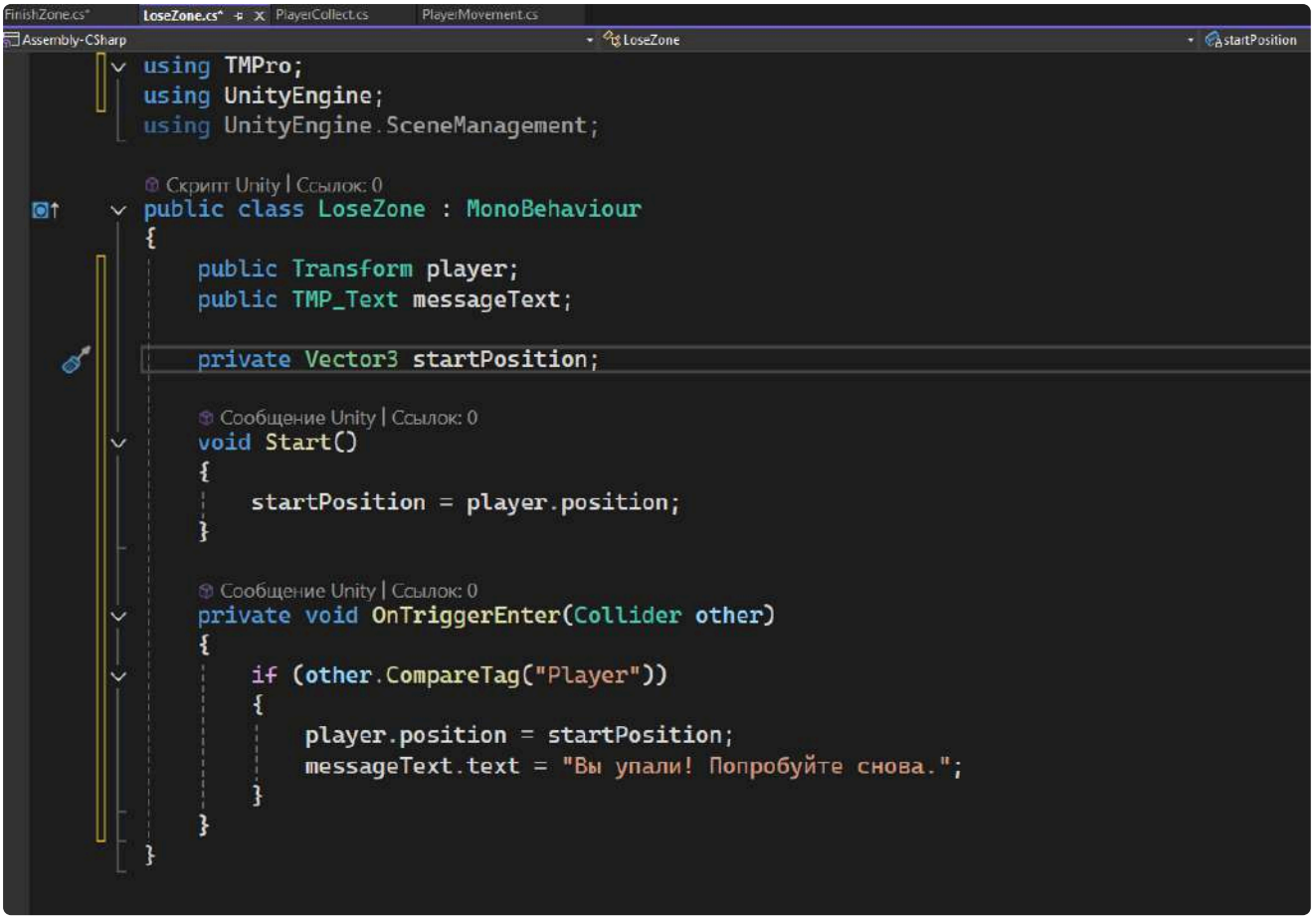


3. Сохраните скрипт. Запустите.
4. Вернитесь в Unity.
5. В поле **Message Text** перетащите объект **MessageText** из Hierarchy.

Альтернативный вариант: возврат игрока на старт вместо перезапуска сцены

Если не хотите перезапускать сцену, можно сделать возврат игрока на стартовую позицию.

Тогда вместо скрипта **LoseZone** используйте такой код:



В Unity нужно будет:

1. Выделить **LoseZone**.
2. В поле **Player** перетащить объект **Player**.
3. В поле **Message Text** перетащить объект **MessageText**.

Возможные ошибки

Игрок не двигается

Проверьте:

```
PlayerMovement добавлен на Player
Rigidbody добавлен на Player
Speed больше 0
Freeze Position не включен
```

Игрок проходит сквозь препятствия

Проверьте:

```
У препятствий есть Box Collider
У Player есть Capsule Collider
У Player есть Rigidbody
```

Зона проигрыша не работает

Проверьте:

```
У LoseZone есть Box Collider
Is Trigger включен
Скрипт LoseZone добавлен на LoseZone
У Player установлен Tag = Player
```

Финиш не работает

Проверьте:

```
У FinishZone есть Box Collider
Is Trigger включен
Скрипт FinishZone добавлен на FinishZone
В поле Message Text добавлен объект MessageText
У Player установлен Tag = Player
```

Тренировочное задание 3. 3D-мини-игра «Лабиринт»

Создать простую 3D-игру, где игрок перемещается по лабиринту, собирает предметы, видит количество собранных объектов на экране и после достижения финиша получает сообщение «**Уровень пройден**».

Создайте новую сцену.

Создание игрового поля

1. В верхнем меню выберите:

```
GameObject → 3D Object → Plane
```

2. Переименуйте объект в:

```
Ground
```

3. Настройте Transform:

```
Position: X = 0, Y = 0, Z = 0
Scale: X = 2, Y = 1, Z = 2
```

Plane будет полом лабиринта.

Создание материала для пола

1. Откройте папку **Materials**.
2. Нажмите:

```
ПКМ → Create → Material
```

3. Назовите материал:

```
GroundMaterial
```

4. Выберите цвет.
5. Перетащите материал на объект **Ground**.

Создание стен лабиринта

Стены будем создавать с помощью объектов **Cube**.

Создание первой стены

1. Выберите:

```
GameObject → 3D Object → Cube
```

2. Переименуйте объект в:

```
Wall_1
```

3. Настройте Transform:

```
Position: X = 0, Y = 0.5, Z = 5
```

```
Scale: X = 10, Y = 1, Z = 0.3
```

Это будет верхняя стена.

Создание остальных стен

Можно дублировать стену через:

```
Ctrl + D
```

Пример расположения стен:

Wall_1

```
Position: X = 0, Y = 0.5, Z = 5
Scale: X = 10, Y = 1, Z = 0.3
```

Wall_2

```
Position: X = 0, Y = 0.5, Z = -5
Scale: X = 10, Y = 1, Z = 0.3
```

Wall_3

```
Position: X = -5, Y = 0.5, Z = 0S
cale: X = 0.3, Y = 1, Z = 10
```

Wall_4

```
Position: X = 5, Y = 0.5, Z = 0
Scale: X = 0.3, Y = 1, Z = 10
```

Это внешние стены лабиринта.

Создание внутренних стен лабиринта

Создайте ещё несколько Cube-объектов.

Wall_5

```
Position: X = -2, Y = 0.5, Z = 2
Scale: X = 0.3, Y = 1, Z = 4
```

Wall_6

```
Position: X = 2, Y = 0.5, Z = -2
Scale: X = 0.3, Y = 1, Z = 4
```

Wall_7

```
Position: X = 0, Y = 0.5, Z = 0
Scale: X = 4, Y = 1, Z = 0.3
```

Wall_8

```
Position: X = -3, Y = 0.5, Z = -2
Scale: X = 3, Y = 1, Z = 0.3
```

Wall_9

```
Position: X = 3, Y = 0.5, Z = 2
Scale: X = 3, Y = 1, Z = 0.3
```

У всех Cube-стен автоматически есть **Box Collider**, поэтому игрок не сможет проходить сквозь стены.

Материал для стен

1. В папке **Materials** создайте новый материал:

```
WallMaterial
```

2. Выберите цвет.
3. Перетащите материал на стены.

Можно выделить сразу несколько стен и назначить материал всем вместе.

Создание игрока

1. Выберите:

```
GameObject → 3D Object → Capsule
```

или:

```
GameObject → 3D Object → Sphere
```

Лучше использовать **Capsule**.

2. Переименуйте объект в:

```
Player
```

3. Настройте Transform:

```
Position: X = -4, Y = 1, Z = -4
Scale: X = 1, Y = 1, Z = 1
```

Если используете Sphere:

```
Position: X = -4, Y = 0.5, Z = -4
```

Добавление Rigidbody игроку

1. Выделите объект **Player**.
2. В Inspector нажмите:

```
Add Component
```

3. Добавьте компонент:

```
Rigidbody
```

4. Настройте Rigidbody:

```
Use Gravity: включено
Is Kinematic: выключено
```

5. В разделе **Constraints** поставьте:

```
Freeze Rotation X
Freeze Rotation Z
```

Это нужно, чтобы игрок не падал на бок.

Не включайте:

```
Freeze Position X
Freeze Position Y
```

```
Freeze Position Z
```

Иначе игрок не сможет двигаться.

Проверка Collider у игрока

Если игрок создан через **Capsule**, у него уже есть:

```
Capsule Collider
```

Если игрок создан через **Sphere**, у него уже есть:

```
Sphere Collider
```

Проверьте, чтобы **Is Trigger** у игрока был выключен.

Создание скрипта движения игрока

Если вы уже делали первое тренировочное задание, можете взять уже готовый скрипт! Он у вас есть! Нажимая на Player - Add Component - и вводите PlayerMovement. Этот скрипт уже имеется у вас в папке Scripts

1. Нажмите в поле иерархии на Player
2. В Inspector справа, в самом низу нажмите кнопку Add Component
3. Начните вводить название скрипта:

PlayerMovement

4. Нажмите Create and Add
5. Сохраните его в вашу созданную папку Scripts
6. Откройте скрипт в Visual Studio

```
using UnityEngine;

Скрипт Unity (ссылок на ресурсы: 3) | Ссылок: 0
public class PlayerMovement : MonoBehaviour
{
    public float speed = 5f;

    private Rigidbody rb;

    Сообщение Unity | Ссылок: 0
    private void Start()
    {
        rb = GetComponent<Rigidbody>();
    }

    Сообщение Unity | Ссылок: 0
    private void Update()
    {
        Vector3 movement = Vector3.zero;

        if (Input.GetKey(KeyCode.W))
            movement += Vector3.forward;

        if (Input.GetKey(KeyCode.S))
            movement += Vector3.back;

        if (Input.GetKey(KeyCode.D))
            movement += Vector3.right;

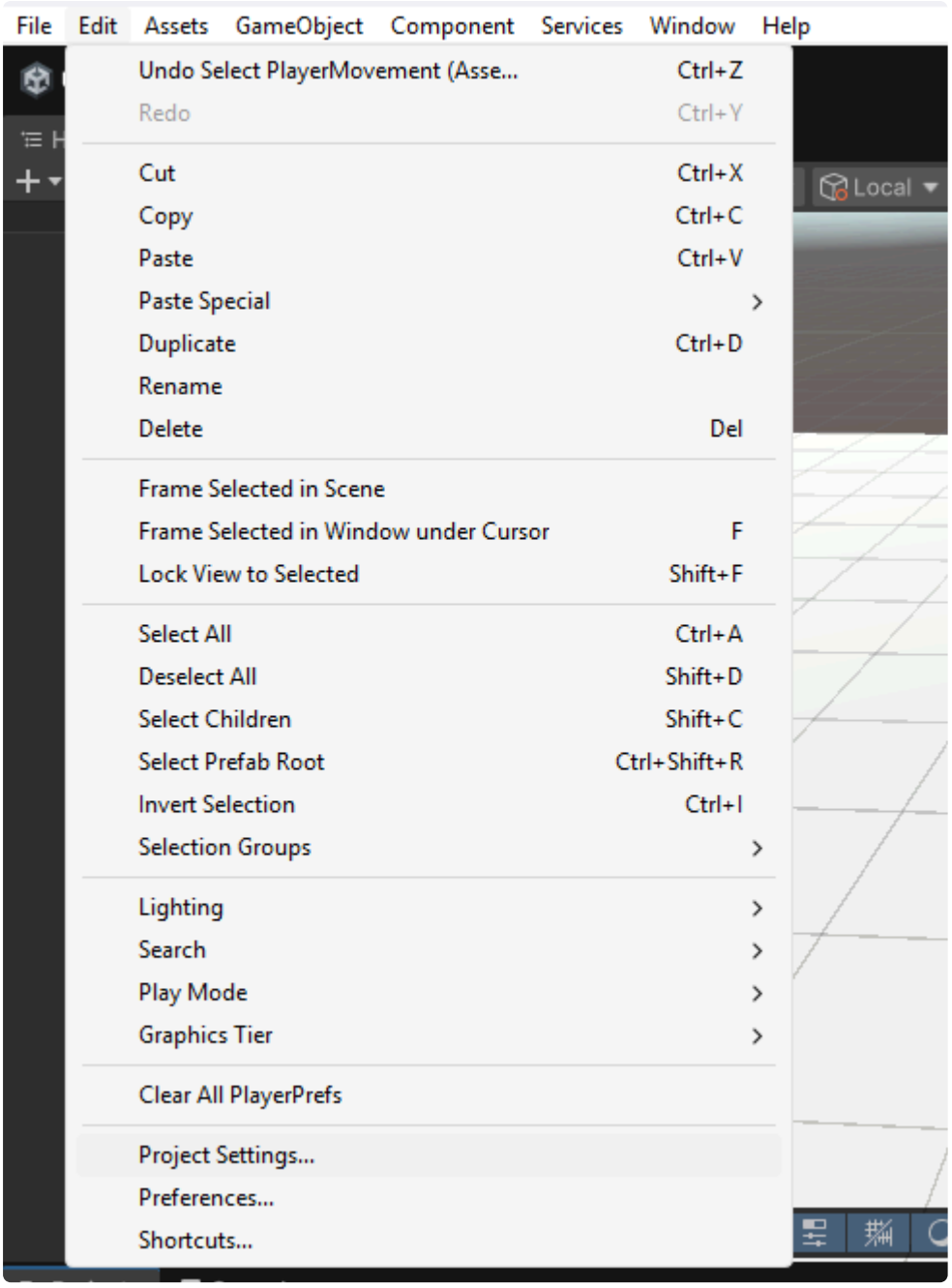
        if (Input.GetKey(KeyCode.A))
            movement += Vector3.left;

        rb.MovePosition(rb.position+movement.normalized * speed * Time.fixedDeltaTime);
    }
}
```

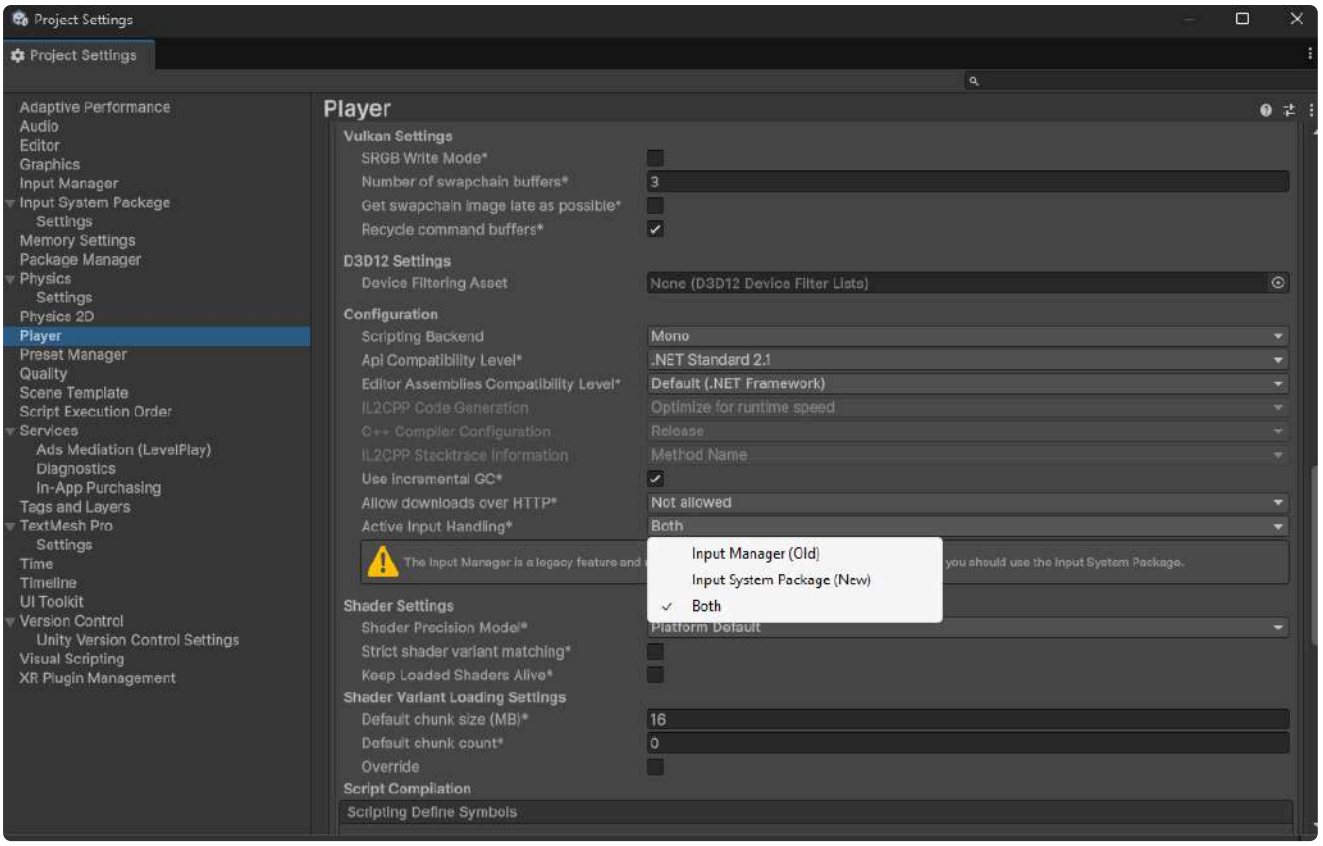
Так выглядит код на передвижение игрока.

Если возникли ошибки в Unity, необходимо в самом Unity изменить Input систему на передвижение. Для этого:

Нажимаем Edit - Project Settings



Ищем слева вкладку Player, внутри Other Setting, листаем до Configuration - в Active Input Handling ставим BOTH



После этого проект перезагружается.

Теперь наш персонаж может двигаться.

Настройка камеры

Для лабиринта удобно сделать камеру сверху.

1. Выделите **Main Camera**.
2. Настройте Transform:

```
Position: X = 0, Y = 12, Z = 0
Rotation: X = 90, Y = 0, Z = 0
```

Камера будет смотреть на лабиринт сверху.

Если хочется под углом:

```
Position: X = 0, Y = 10, Z = -8
Rotation: X = 55, Y = 0, Z = 0
```

Создание собираемого объекта

1. Выберите:

```
GameObject → 3D Object → Sphere
```

2. Переименуйте объект в:

```
Collectible
```

3. Настройте Transform:

```
Position: X = -3, Y = 0.5, Z = -2
Scale: X = 0.4, Y = 0.4, Z = 0.4
```

4. У объекта есть **Sphere Collider**.
5. В компоненте **Sphere Collider** включите:

```
Is Trigger
```

Материал для собираемых объектов

1. В папке **Materials** создайте материал:

```
CollectibleMaterial
```

2. Выберите яркий цвет.
3. Перетащите материал на объект **Collectible**.

Назначение тега Collectible

1. Выделите объект **Collectible**.
2. В Inspector сверху найдите поле **Tag**.
3. Нажмите:

```
Add Tag
```

4. Нажмите плюс.
5. Создайте тег:

```
Collectible
```

6. Вернитесь к объекту **Collectible**.
7. Назначьте ему тег:

```
Collectible
```

Создание префаба

1. Откройте папку **Prefabs**.
2. Перетащите объект **Collectible** из **Hierarchy** в папку **Prefabs**.

Теперь это префаб.

Размещение 5 собираемых объектов

Создайте копии предмета:

```
Ctrl + D
```

Разместите минимум 5 объектов внутри лабиринта.

Пример координат:

Collectible_1

```
Position: X = -3, Y = 0.5, Z = -2
```

Collectible_2

```
Position: X = 3, Y = 0.5, Z = -3
```

Collectible_3

```
Position: X = -1, Y = 0.5, Z = 3
```

Collectible_4

```
Position: X = 4, Y = 0.5, Z = 1
```

Collectible_5

```
Position: X = 1, Y = 0.5, Z = 4
```

Важно: у всех объектов должно быть:

```
Tag = Collectible
Is Trigger = включен
```

Создание UI-счетчика

1. Выберите:

```
GameObject → UI → Text – TextMeshPro
```

2. Если Unity предложит импортировать TMP Essentials, нажмите:

```
Import TMP Essentials
```

3. Переименуйте объект в:

```
ScoreText
```

4. В поле **Text** напишите:

```
Собрано: 0 / 5
```

5. Настройте расположение сверху слева

Создание текста победы

1. Выберите:

```
GameObject → UI → Text – TextMeshPro
```

2. Переименуйте объект в:

```
MessageText
```

3. В поле Text напишите:

```
Уровень пройден
```

4. Расположите по центру экрана
5. Отключите объект **MessageText**:

В Inspector снимите галочку рядом с названием объекта.

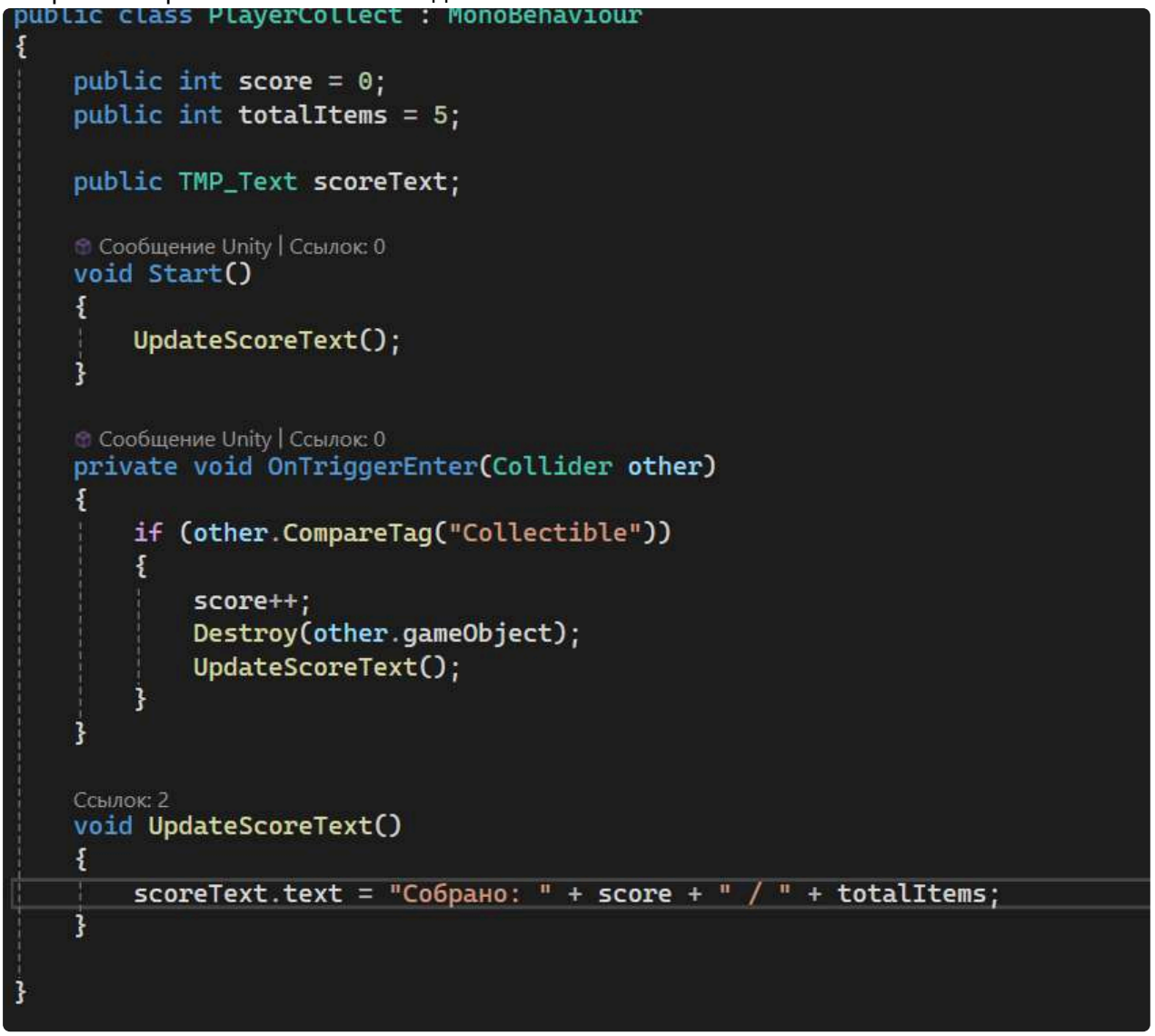
Он должен появиться только после достижения финиша.

Создание скрипта сбора предметов

1. Нажмите на Player.
2. Справа внизу Add Component - и начните вводить название скрипта:

```
PlayerCollector
```

3. Откройте скрипт и напишите код:



4. Сохраните скрипт.
5. Вернитесь в Unity.
6. Перетащите скрипт **PlayerCollector** на объект **Player**.
7. В поле **Score Text** перетащите объект **ScoreText**.
8. В поле **Total Items** укажите:

```
5
```

Создание финишной зоны

1. Выберите:

```
GameObject → 3D Object → Cube
```

2. Переименуйте объект в:

```
FinishZone
```

3. Разместите его в конце лабиринта:

```
Position: X = 4, Y = 0.25, Z = 4Scale: X = 1.5, Y = 0.2, Z = 1.5
```

4. В компоненте **Box Collider** включите:

```
Is Trigger
```

5. Создайте материал **FinishMaterial** и назначьте его финишной зоне.

Создание скрипта финиша

1. Нажмите на FinishZone.
2. Справа внизу Add Component - и начните вводить название скрипта:

```
FinishZone
```

3. Откройте скрипт и напишите код:



- Сохраните скрипт.
- Вернитесь в Unity.

- Перетащите скрипт **FinishZone** на объект **FinishZone**.
- В поле **Message Text** перетащите объект **MessageText**.

Назначение тега Player

1. Выделите объект **Player**.
2. В Inspector сверху найдите **Tag**.
3. Выберите:

```
Player
```

Если тега нет:

```
Tag → Add Tag → + → Player
```

Потом снова выберите Player и назначьте ему тег **Player**.

Проверка всех компонентов

У Player должно быть:

```
Transform
Capsule Collider
Rigidbody
PlayerMovement
PlayerCollector
Tag = Player
```

У стен должно быть:

```
Box Collider
Is Trigger выключен
```

У собираемых объектов должно быть:

```
Sphere Collider
Is Trigger включен
Tag = Collectible
```

У FinishZone должно быть:

```
Box Collider
Is Trigger включен
```

```
FinishZone Script
```

Проверка в Play Mode

Нажмите **Play**.

Проверьте:

1. Игрок двигается клавишами:

```
W A S D
```

2. Игрок не проходит сквозь стены.
3. Предметы исчезают при касании.
4. Счетчик увеличивается:

```
Собрано: 1 / 5
Собрано: 2 / 5...
```

5. При достижении финиша появляется сообщение:

```
Уровень пройден
```

Возможные ошибки

Игрок не двигается

Проверьте:

```
PlayerMovement добавлен на Player
Rigidbody добавлен на Player
Speed больше 0
Freeze Position не включен
```

Игрок летает или странно отталкивается

Проверьте:

```
Freeze Rotation X включен
Freeze Rotation Z включен
Freeze Position выключен
```

Можно увеличить массу Rigidbody:

```
Mass = 1
```

И убедиться, что стартовая позиция игрока над полом:

```
Capsule: Y = 1
Sphere: Y = 0.5
```

Игрок проходит сквозь стены

Проверьте:

```
У стен есть Box Collider
У стен Is Trigger выключен
У Player есть Collider
У Player есть Rigidbody
```

Предметы не собираются

Проверьте:

```
У предметов включен Is Trigger
У предметов Tag = Collectible
У Player есть Rigidbody
У Player есть PlayerCollector
```

Сообщение «Уровень пройден» не появляется

Проверьте:

```
У FinishZone включен Is Trigger
У Player стоит Tag = Player
Скрипт FinishZone добавлен на FinishZone
MessageText перетащен в поле Message Text
```

Тренировочное задание 4. Главное меню и переход между сценами»

Создать проект Unity, в котором есть:

- сцена главного меню;
- игровая сцена;

- кнопка **Start** для перехода в игру;
- кнопка **Exit** для выхода из приложения;
- кнопка **Restart** на игровой сцене для перезапуска уровня;

В папке Scenes создайте сцены MainMenu и GameScene

Добавление сцен в Build Settings

Это очень важный шаг. Без него переход между сценами работать не будет.

1. Откройте:

```
File → Build Settings
```

2. Нажмите кнопку:

```
Add Open Scenes
```

Сейчас добавится открытая сцена **GameScene**.

3. Потом откройте сцену **MainMenu**:

```
Project → Scenes → MainMenu
```

4. Снова откройте:

```
File → Build Settings
```

5. Нажмите:

```
Add Open Scenes
```

Теперь в списке должны быть обе сцены:

```
MainMenuGameScene
```

Важно: **MainMenu должна быть первой**, то есть под номером `0`.

Если порядок неправильный, перетащите **MainMenu** выше **GameScene**.

Должно быть так:

```
0 MainMenu
```

```
1 GameScene
```

Создание Canvas на сцене MainMenu

1. Откройте сцену **MainMenu**.
2. В верхнем меню выберите:

```
GameObject → UI → Canvas
```

Unity автоматически создаст:

```
CanvasEventSystem
```

Если **EventSystem** не появился, его можно создать вручную:

```
GameObject → UI → Event System
```

Настройка Canvas

1. Выделите объект **Canvas**.
2. В Inspector найдите компонент **Canvas Scaler**.
3. Настройте:

```
UI Scale Mode: Scale With Screen Size
Reference Resolution: X = 1920, Y = 1080
Screen Match Mode: Match Width Or Height
Match: 0.5
```

Так интерфейс будет нормально отображаться при разных размерах экрана.

Добавление названия игры

1. На сцене **MainMenu** выберите:

```
GameObject → UI → Text – TextMeshPro
```

Если появится окно импорта, нажмите:

```
Import TMP Essentials
```

2. Переименуйте объект в:

```
TitleText
```

3. В Inspector в поле **Text Input** напишите, например:

```
3D Лабиринт
```

или:

```
Полоса препятствий
```

4. Настройте текст:

```
Font Size = 70
Alignment = Center
```

5. В Rect Transform задайте примерное положение:

```
Anchor: Middle Center
Pos X = 0
Pos Y = 250
Width = 800
Height = 120
```

Добавление кнопки Start

1. Выберите:

```
GameObject → UI → Button – TextMeshPro
```

2. Переименуйте кнопку в:

```
StartButton
```

3. Внутри кнопки будет дочерний объект с текстом.

Откройте кнопку в Hierarchy:

```
StartButton → Text (TMP)
```

4. Выделите текст и напишите:

```
Start
```

или:

```
Начать игру
```

5. Настройте положение кнопки **StartButton**:

```
Anchor: Middle Center
Pos X = 0
Pos Y = 50
Width = 300
Height = 80
```

Добавление кнопки Exit

1. Создайте вторую кнопку:

```
GameObject → UI → Button – TextMeshPro
```

2. Переименуйте её в:

```
ExitButton
```

3. В дочернем тексте кнопки напишите:

```
Exit
```

или:

```
Выход
```

4. Настройте положение:

```
Anchor: Middle Center
Pos X = 0
Pos Y = -70
Width = 300
Height = 80
```

Создание скрипта для главного меню

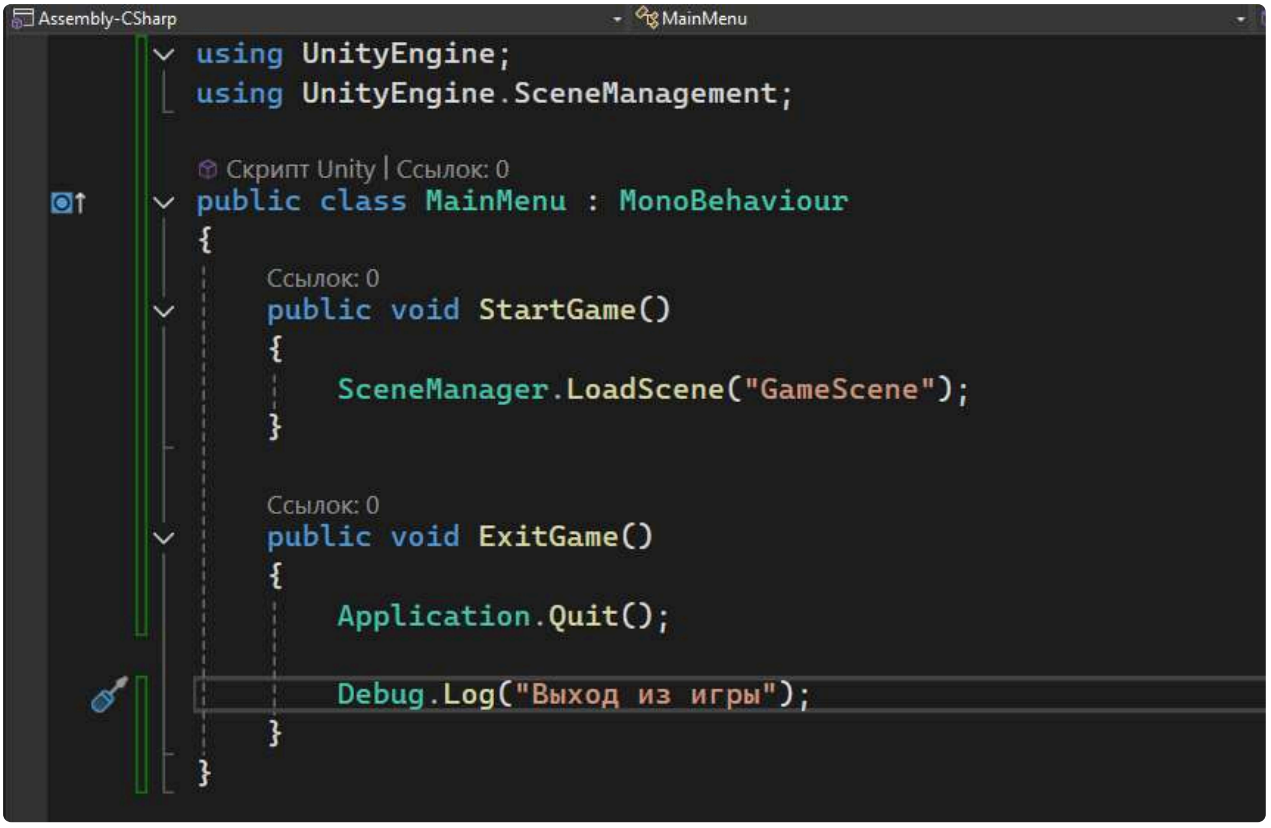
1. Откройте папку **Scripts**.
2. Нажмите:

```
ПКМ → Create → Sctipting → C# Script
```

3. Назовите скрипт:

```
MainMenu
```

4. Откройте скрипт.
5. Напишите код:



Создание объекта для скрипта меню

1. Вернитесь в Unity.
2. На сцене **MainMenu** создайте пустой объект:

```
GameObject → Create Empty
```

3. Переименуйте его в:

```
MenuManager
```

4. Перетащите скрипт **MainMenu** на объект **MenuManager**.

Подключение кнопки Start к скрипту

1. Выделите кнопку **StartButton**.
2. В Inspector найдите компонент **Button**.
3. Внизу найдите блок:

```
On Click()
```

4. Нажмите плюс:

```
+
```

5. В появившееся пустое поле перетащите объект:

```
MenuManager
```

6. Справа в выпадающем списке выберите:

```
MainMenu → StartGame()
```

Теперь кнопка Start будет открывать игровую сцену.

Подключение кнопки Exit к скрипту

1. Выделите кнопку **ExitButton**.
2. В компоненте **Button** найдите:

```
On Click()
```

3. Нажмите плюс:

```
+
```

4. Перетащите объект **MenuManager** в появившееся поле.
5. В выпадающем списке выберите:

```
MainMenu → ExitGame()
```

Важно: в редакторе Unity кнопка Exit не закроет Play Mode через `Application.Quit()`.
Но сообщение появится в Console:

```
Выход из игры
```

В готовом Build кнопка Exit будет закрывать приложение.

Создание простой игровой сцены GameScene

1. Откройте сцену:

```
Project → Scenes → GameScene
```

2. Добавьте пол:

```
GameObject → 3D Object → Plane
```

3. Переименуйте его в:

```
Ground
```

4. Настройте Scale:

```
Scale X = 2
Scale Y = 1
Scale Z = 2
```

5. Добавьте игрока:

```
GameObject → 3D Object → Capsule
```

6. Переименуйте:

```
Player
```

7. Настройте Position:

```
X = 0Y = 1Z = 0
```

8. Добавьте Rigidbody:

```
Player → Add Component → Rigidbody
```

9. В Rigidbody включите:

```
Freeze Rotation X
Freeze Rotation Z
```

Добавление движения игрока на GameScene

В папке **Scripts** создайте или используйте уже готовый скрипт **PlayerMovement**.

Настройка камеры на GameScene

1. Выделите **Main Camera**.
2. Настройте Transform:

```
Position: X = 0, Y = 8, Z = -8Rotation: X = 45, Y = 0, Z = 0
```

3. Проверьте, что камера видит игровую сцену.

Добавление кнопки Restart на игровой сцене

1. Убедитесь, что открыта сцена **GameScene**.
2. Создайте Canvas:

```
GameObject → UI → Canvas
```

Если Canvas уже есть, второй создавать не нужно.

3. Если EventSystem не создался автоматически:

```
GameObject → UI → Event System
```

4. Создайте кнопку:

```
GameObject → UI → Button – TextMeshPro
```

5. Переименуйте кнопку в:

```
RestartButton
```

6. Внутри кнопки измените текст на:

```
Restart
```

или:

```
Заново
```

7. Настройте положение кнопки:

```
Anchor: Top Right
Pos X = -120
Pos Y = -60
Width = 220
Height = 70
```

Создание скрипта для игровой сцены

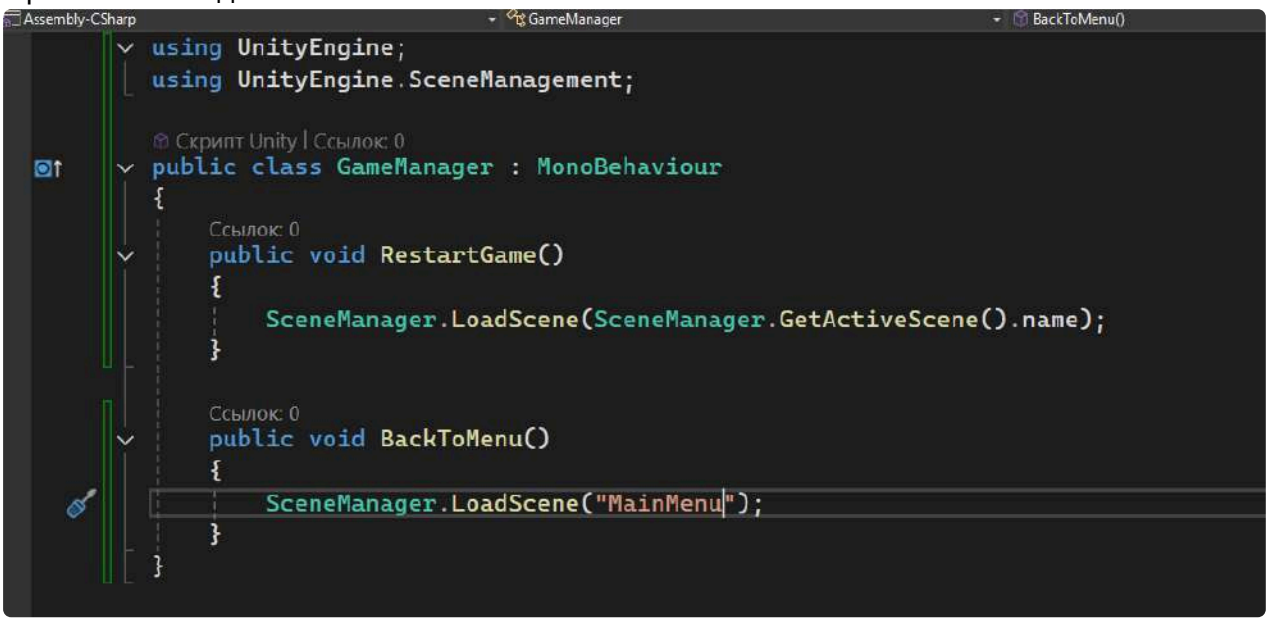
1. На сцене **GameScene** создайте пустой объект:

```
GameObject → Create Empty
```

2. Переименуйте его в:

```
GameManager
```

3. Добавьте на него скрипт GameManager через Add Component
4. Пропишите код:



Подключение кнопки Restart

1. Выделите **RestartButton**.
2. В Inspector найдите компонент **Button**.
3. В блоке **On Click()** нажмите:

```
+
```

4. Перетащите объект **GameManager** в пустое поле.
5. В выпадающем списке выберите:

```
GameManager → RestartGame()
```

Теперь кнопка Restart будет перезапускать игровую сцену.

Дополнительно: кнопка возврата в меню

Это не обязательно, но удобно для проверки.

1. Создайте ещё одну кнопку:

```
GameObject → UI → Button – TextMeshPro
```

2. Назовите:

```
MenuButton
```

3. Текст кнопки:

```
Menu
```

или:

```
В меню
```

4. В **On Click()** подключите объект **GameManager**.
5. Выберите функцию:

```
GameManager → BackToMenu()
```

Проверка перехода из меню в игру

1. Откройте сцену **MainMenu**.
2. Нажмите **Play**.
3. Нажмите кнопку:

```
Start
```

4. Должна открыться сцена:

```
GameScene
```

Если сцена не открывается, проверьте:

```
GameScene добавлена в Build Settings
Название сцены в коде написано точно: "GameScene"
Нет ошибок в Console
```

Проверка кнопки Restart

1. На сцене **GameScene** нажмите **Play**.
2. Подвиньте игрока с места.
3. Нажмите кнопку:

```
Restart
```

4. Сцена должна перезапуститься, а игрок вернуться на стартовую позицию.

Проверка кнопки Exit

1. Откройте сцену **MainMenu**.
2. Нажмите **Play**.
3. Нажмите кнопку:

```
Exit
```

В редакторе Unity игра не закроется, но в Console должно появиться:

```
Выход из игры
```

После Build кнопка будет закрывать приложение.

Возможные ошибки

Кнопки не нажимаются

Проверьте:

```
На сцене есть EventSystem
Кнопки находятся внутри Canvas
Canvas активен
В On Click() добавлен объект со скриптом
Выбрана нужная функция
```

Start не открывает GameScene

Проверьте:

```
Сцена GameScene добавлена в Build Settings
Название сцены в коде совпадает с названием файла сцены
В коде написано SceneManager.LoadScene("GameScene");
```

Название должно совпадать полностью:

```
GameScene
```

Например, если сцена называется `Game Scene`, а в коде `GameScene`, переход не сработает.

Restart не работает

Проверьте:

```
GameManager есть на сцене GameScene
Скрипт GameManager висит на объекте GameManager
Кнопка Restart подключена к функции RestartGame()
Сцена добавлена в Build Settings
```

В Console ошибка про SceneManagement

Проверьте, что в коде сверху есть строка:

```
using UnityEngine.SceneManagement;
```